



Article

Malware Detection Based on API Call Sequence Analysis: A Gated Recurrent Unit–Generative Adversarial Network Model Approach

Nsikak Owoh ^{1,*}, John Adejoh ², Salaheddin Hosseinzadeh ¹, Moses Ashawa ¹, Jude Osamor ³ and Ayyaz Qureshi ¹

¹ Department of Cyber Security and Networks, Glasgow Caledonian University, Glasgow G4 0BA, UK; salaheddin.hosseinzadeh@gcu.ac.uk (S.H.); moses.ashawa@gcu.ac.uk (M.A.); aqu2@gcu.ac.uk (A.Q.)

² Department of Software Engineering, African University of Science and Technology, Abuja 900107, Nigeria; ajohn@staff.aust.edu.ng

³ School of Computer Science and Engineering, University of Westminster, 309 Regent Street, London W1B 2HW, UK; j.osamor@westminster.ac.uk

* Correspondence: nsikak.owoh@gcu.ac.uk

Abstract: Malware remains a major threat to computer systems, with a vast number of new samples being identified and documented regularly. Windows systems are particularly vulnerable to malicious programs like viruses, worms, and trojans. Dynamic analysis, which involves observing malware behavior during execution in a controlled environment, has emerged as a powerful technique for detection. This approach often focuses on analyzing Application Programming Interface (API) calls, which represent the interactions between the malware and the operating system. Recent advances in deep learning have shown promise in improving malware detection accuracy using API call sequence data. However, the potential of Generative Adversarial Networks (GANs) for this purpose remains largely unexplored. This paper proposes a novel hybrid deep learning model combining Gated Recurrent Units (GRUs) and GANs to enhance malware detection based on API call sequences from Windows portable executable files. We evaluate our GRU–GAN model against other approaches like Bidirectional Long Short-Term Memory (BiLSTM) and Bidirectional Gated Recurrent Unit (BiGRU) on multiple datasets. Results demonstrated the superior performance of our hybrid model, achieving 98.9% accuracy on the most challenging dataset. It outperformed existing models in resource utilization, with faster training and testing times and low memory usage.

Keywords: malware detection; dynamic malware analysis; API call sequence; deep learning; Gated Recurrent Unit; Generative Adversarial Network



Citation: Owoh, N.; Adejoh, J.; Hosseinzadeh, S.; Ashawa, M.; Osamor, J.; Qureshi, A. Malware Detection Based on API Call Sequence Analysis: A Gated Recurrent Unit–Generative Adversarial Network Model Approach. *Future Internet* **2024**, *16*, 369. <https://doi.org/10.3390/fi16100369>

Academic Editors: Thomas Loruenser and Stephan Krenn

Received: 2 September 2024

Revised: 9 October 2024

Accepted: 10 October 2024

Published: 13 October 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In recent years, the cybersecurity landscape has undergone a significant evolution, driven by the exponential growth of cyber threats. This proliferation of malicious activities has underscored the urgent need for innovative approaches to detect and mitigate potential attacks [1]. The swift progress of technology, despite its many benefits, has made computer systems and networks more susceptible to a wide range of threats [2]. Malware, a term derived from “malicious software”, represents one of the most significant challenges in this domain. By the first quarter of 2024, over 1.3 billion unique malware samples had been recorded, marking a steady increase from the approximately 1.23 billion documented by the end of 2021 [3]. This alarming trend highlights the critical importance of developing robust cybersecurity solutions to combat these ever-evolving threats [4].

Malware can be classified into various categories, including viruses, worms, trojans, spyware, ransomware, and logic bombs, each with distinct behavioral characteristics and attributes [5]. The potential impact of these malicious programs ranges from unauthorized

access and service disruption to data theft and sometimes irreparable system damage [6,7]. Understanding the infiltration methods of malware into computer systems requires knowledge of malware families, as members of the same family mostly exhibit comparable behaviors. Malware analysts and incident responders can enhance preventive measures against malware by identifying its family and also simplifying the overall analysis process. Malware analysis, an important aspect of cybersecurity, involves the systematic investigation and dissection of malicious software. This process provides valuable insights into the operational mechanisms, objectives, and potential impacts of these programs on computer systems, networks, and end-user devices [8]. By analyzing the tactics, techniques, and procedures (TTPs) employed by malicious actors, cybersecurity professionals can develop more effective countermeasures, including intrusion detection system (IDS) rules, antivirus signatures, and defensive strategies [2].

Malware analysis typically employs three primary methods: static, dynamic, and hybrid approaches. Static analysis examines the structure of the malware code without execution, while dynamic analysis observes the behavior of the malware by running it in a controlled environment. Hybrid techniques combine elements of both approaches [9]. One critical aspect of dynamic analysis is the examination of API (Application Programming Interface) calls made by the malware, as these represent its behavioral patterns and interactions with the operating system [10]. The Windows API, consisting of numerous functions within specific software libraries, plays a key role in facilitating interactions between applications and the operating system. Analyzing the patterns and sequences of API calls is essential for verifying the legitimacy of system actions and identifying potentially malicious behavior [11]. An application makes several API calls to perform tasks. For instance, to delete a file, the application will call the DeleteFileA Windows API [12–14].

Recently, deep learning algorithms have evolved as useful tools in the field of malware detection and classification. These algorithms, a subset of machine learning techniques, leverage artificial neural network architectures to discover and learn complex patterns from large, high-dimensional datasets [15,16]. Deep learning models can automatically extract high-level features and learn from both labeled and unlabeled data, producing highly accurate results with low false-positive rates [17].

Although numerous studies have advanced deep learning for malware detection using API calls, the benefits of Generative Adversarial Networks (GANs) in this area remain largely unexplored. Generative Adversarial Networks have emerged as a powerful tool in cybersecurity, serving many roles in both improving detection methodologies and simulating potential threats. In enhancing detection models, GANs can generate synthetic malware samples, thereby creating more robust and diverse training datasets [18]. This capability is particularly valuable in improving the detection of novel malware variants. Furthermore, GANs play a crucial role in adversarial training, simulating attacks on detection systems to identify and address weaknesses, ultimately leading to more resilient malware detection frameworks.

Another significant application of GANs in this field is anomaly detection. By learning the distribution of normal API call patterns, GANs can effectively identify deviations that may indicate potential malware activity [19,20]. This approach has shown particular promise in the detection of zero-day threats. Additionally, GANs can be employed to generate new malware variants for research purposes, aiding in proactive defense by anticipating future attack vectors. GANs offer unique advantages such as generating synthetic API call sequences to augment training datasets, enhancing system robustness through adversarial training, enabling unsupervised anomaly detection, adapting dynamically to evolving threats, extracting complex features, improving cross-family malware detection, and reducing false-positive rates by refining the distinction between legitimate and malicious behavior. When combined with algorithms like Gated Recurrent Units (GRUs), GANs can significantly improve detection accuracy while keeping computational overhead low [21]. This hybrid approach leverages GRU efficiency in processing sequential data and GAN generative capabilities, creating a powerful, resource-efficient malware detection system.

Building on the potential of GANs in malware detection using API calls, this paper presents a hybrid deep learning model to tackle the proliferation of sophisticated malware. Our study combines the capabilities of GANs with advanced algorithms like GRUs to enhance the accuracy and computational efficiency of malware analysis based on API calls from portable executable (PE) files. The hybrid model aims to improve detection accuracy while maintaining low computational overhead.

Our paper makes several key contributions to the field of malware detection for Windows environments:

1. We conduct a comprehensive evaluation of different deep learning algorithms, including BiLSTM, BiGRU, and GRU-GAN, for the classification and detection of malware from API call sequences.
2. We propose a hybrid model that combines GRU and GAN algorithms for the accurate detection of malware from API call sequences.
3. We provide an in-depth analysis of model performance across diverse datasets to test the generalization capabilities of the models.
4. We benchmark the proposed hybrid model against existing models, focusing on both performance and computational overhead.

The structure of this paper is as follows: Section 2 explores Windows API calls and deep learning algorithms and reviews related work on malware detection using API calls. Section 3 presents the proposed model architecture and methodology. In Section 4, we analyze the results, evaluate the performance of the proposed model, and assess its computational efficiency. We also benchmark our model against existing malware detection models. Finally, Section 5 concludes the paper and outlines potential directions for future research.

2. Materials and Methods

This section provides an overview of the Windows application programming interface. Additionally, it explores the application of machine learning and deep learning algorithms in malware detection from API calls.

2.1. Windows API Calls

API call sequence analysis is a critical technique for understanding program behavior in Windows systems. Windows applications rely heavily on API functions, making the Win32 API an important component [22]. However, these same API functions can be exploited by malware, serving as entry points for attacks. API call sequences, obtained through dynamic analysis in controlled environments, reveal behavioral patterns of both benign and malicious programs.

Malware often creates or manipulates processes and threads to execute malicious code or maintain persistence. This is often achieved by making API calls such as `CreateProcess()`, `CreateThread()`, `OpenProcess()`, `TerminateProcess()`, and `SuspendThread()/ResumeThread()`. The data extracted from these API calls typically includes process and thread handles, identifiers, and execution status information, which can provide valuable insights into malware behavior and operations. Also, malware frequently uses memory manipulation to inject code, hide its presence, or access sensitive data. Important API calls include `VirtualAlloc()`, `VirtualProtect()`, `WriteProcessMemory()`, and `ReadProcessMemory()`. Memory addresses, memory protection flags, and content of memory regions are some of the data retrieved from these API calls. Malicious PEs often interact with the file system to persist, spread, or access sensitive information [8]. Some API calls made in this case are, `CreateFile()`, `WriteFile()/ReadFile()`, `CopyFile()`, `DeleteFile()` and `SetFileAttributes()`. The extracted data include file handles, file content, file attributes, and error codes indicating the success or failure of operations.

Additionally, the Windows Registry is a common target for malware seeking to persist or modify system settings. Some of the essential API calls include `RegOpenKeyEx()`, `RegSetValueEx()`, `RegQueryValueEx()`, and `RegDeleteValue()`. These API calls enable malware to access, modify, and delete registry keys and values, allowing them to alter system behavior

and evade detection. The data retrieved from these API calls typically includes registry key handles, registry values and data, and error codes related to registry operations. Similarly, malicious PEs often establish network connections to receive commands, exfiltrate data, or propagate to other systems. Some of the API calls used include `socket()`, `connect()`, `send()/recv()`, and `gethostbyname()`. These API calls enable malware to create network sockets, connect to remote systems, transmit and receive data, and resolve hostnames to IP addresses [23]. The data retrieved from these API calls typically includes socket descriptors, IP addresses, and port numbers, network data (which may contain command-and-control communications or exfiltrated data), and error codes related to network operations. It is important to note that benign executables may also utilize these API calls; however, the specific sequence of calls, with other attributes such as file metadata, entropy, and import table, can aid in distinguishing malicious PEs from benign ones.

This dynamic approach offers deeper insights into malware activities compared to static code analysis alone [24]. By leveraging this technique, researchers and security professionals can build more effective malware detection systems, significantly enhancing the security of Windows-based systems. A multi-layered defense strategy, however, would include API call monitoring, secure network communications, regular updates, logging, and auditing to secure Windows systems against malicious executables further.

2.2. Deep Learning Algorithms

Deep learning, a branch of machine learning, uses complex artificial neural networks to find detailed patterns and features in data. These advanced algorithms are particularly effective at handling large, high-dimensional datasets and can automatically extract high-level features efficiently [25]. They are valuable for working with both labeled and unlabeled data, which is useful in fields like malware analysis, where labeled data (malicious or attack traffic) can be difficult to obtain.

One of the main strengths of deep learning algorithms is their ability to achieve high accuracy with low false-positive rates. This low false-positive rate is particularly important in malware analysis because false alarms can be costly and time-consuming. Deep learning networks have many hidden layers, which allow them to learn features hierarchically: lower layers capture basic features, while higher layers identify more abstract concepts [26]. This layered approach helps deep learning models uncover complex patterns that shallow machine learning algorithms or human analysts might miss. This capability is particularly useful for tasks like malware detection, where it can identify subtle signs of malicious behaviors in large datasets. In malware analysis, deep learning algorithms can extract meaningful patterns from data, helping detect malware and categorize it into different families. A novel use of this technology is analyzing the sequence of API calls made by malicious code, which acts as a behavioral fingerprint of its actions within the system [27,28]. This approach leverages the ability of deep learning to interpret complex sequences, providing a powerful tool for identifying and classifying malware based on behavior rather than just static characteristics.

2.3. Related Work

In related work, Almaleh et al. [2] proposed a hybrid machine and deep learning model for malware detection from API call sequences. It uses logistic regression and RNN models on a large dataset (API-Call-Sequences) of over 40,000 malicious and 1000 benign API call sequences. The hybrid model achieved 84% accuracy on the balanced dataset and 98% on the imbalanced dataset, outperforming baselines. However, computational requirements, including training and testing time and the memory usage of the models, were not discussed. Similarly, in [4], the authors used machine learning and deep learning algorithms for malware detection and classification, comparing their performance using datasets like API-Call-Sequences and APIMDS. The system employs a two-step approach: a Malware Detection algorithm identifies malware and a Malware Classification algorithm categorizes it based on API call sequences. Algorithms tested included Random Forest, CatBoost, XGBoost, ExtraTrees, TabNet, Bi-LSTM, and BiGRU. The BiGRU algorithm

showed superior performance in Recall, AUC-ROC, F1 score, and accuracy for malware detection, while the ExtraTrees algorithm excelled in malware classification. BiGRU was particularly effective in detecting malicious files, whereas ExtraTrees better discriminated between malware types.

Gençaydin et al. [6] introduced the VirusShare and VirusSample datasets for benchmarking malware classification models using static and dynamic analysis to extract API calls. They preprocess the data by removing families with fewer than 100 samples and balancing it with an upper limit of 300 samples per family. The study benchmarks models like SVM, XGBoost, and CANINE, comparing performance on imbalanced and balanced datasets. While providing insights, the datasets may lack diversity, impacting generalizability. The paper also does not detail the computational resources, such as the training and testing time of the models.

In [11], a behavioral malware detection method using Deep Graph Convolutional Neural Networks (DGCNNs) is proposed, involving an eight-step process. The approach analyzes Portable Executable (PE) files in a Cuckoo Sandbox, extracts API call sequences, and creates behavioral graphs for binary classification. Using the API-Call-Sequences dataset, the study finds that LSTM outperforms two DGCNN models in detecting malware. While the AUC metric highlights the models' effectiveness, PCA visualization shows distinct classification behaviors among different architectures. The study notes a need for additional regularization techniques to address observed overfitting after ten epochs.

Authors in [14] employed Long Short-Term Memory (LSTM) for malware classification by analyzing Windows API call sequences. They used the Mal-API-2019 dataset of 7107 malware samples. Treating malware detection as a text classification problem, their LSTM-based model achieved an accuracy between 83.5% and 98.5%, with an overall accuracy of up to 98.50%. The study reported minimal model complexity, and the method was robust against class imbalance. However, the model recorded a relatively high training time with no testing time and memory usage recorded.

Further, Maniriho et al. [29] proposed API-MalDetect, a deep learning-based framework for detecting malware in Windows systems. It employs an NLP-based encoder for API calls and a hybrid feature extractor combining CNNs and BiGRU to process API call sequences. API-MalDetect aims to detect new malware attacks and reduce bias during training and testing. The authors reveal that experimental results outperform existing techniques in accuracy, precision, recall, F1 score, and AUC on benchmark datasets. The framework also highlights key API calls for malware identification. Nevertheless, the memory usage of the models on the three datasets tested was not presented.

In [30], the authors developed a malware detection system by employing Bidirectional Long Short-Term Memory (BiLSTM) to analyze API calls. Using the Cuckoo sandbox for API call extraction, they processed a dataset of 21,378 samples. The authors compared various models, including GRU, BiGRU, LSTM, BiLSTM, and SimpleRNN, with BiLSTM achieving the highest accuracy of 97.85% in malware detection. The results demonstrated the superiority of BiLSTM, highlighting the effectiveness of deep learning techniques for robust malware detection, though there is a need for improvement in evaluation metrics results. Kim et al. [31] presented a dynamic malware analysis method using DNA sequence alignment algorithms, specifically multiple sequence alignment (MSA) and longest common subsequences (LCS), to identify common API call sequence patterns in malware. Analyzing 23,080 samples from the Malicia project and VirusTotal, they developed a signature database and an API-based malware detection system (APIMDS), which showed high accuracy and low error rates. The study highlighted the benefits of dynamic analysis and system adaptability to various devices. However, the computational complexity of the model was not explored.

In addition, [32] proposed an algorithm for detecting ransomware by analyzing network traffic. Tested on 19 ransomware families, it detected activity in under 20 s and prevented the loss of more than ten files. The algorithm also facilitated file recovery using stored network traffic. Analytical models estimated early detection and false alarm

probabilities, with tests showing a low false alarm rate. However, the scope of the study was limited to 19 ransomware families, potentially missing other behaviors, and it did not address ransomware detected through API calls.

In [33], a two-stage mixed ransomware detection model that combines a Markov model and a Random Forest model is introduced. The first stage analyzes Windows API call sequence patterns using the Markov model to capture ransomware characteristics. In the second stage, a Random Forest model is employed to manage false positive and false negative rates effectively. This method achieved an overall accuracy of 97.3%, with a false positive rate of 4.8% and a false negative rate of 1.5%. The study underscores the importance of a two-stage approach, leveraging both Markov chain and machine learning techniques. Despite its high accuracy, there is potential for further improvement. In [34], the Markhor method is developed, utilizing system call data and control dependency sequences to detect a weighted list of malicious patterns. By analyzing these patterns, the method identifies malicious processes. It uses a fuzzy algorithm to assess the similarity of file system call sequences to known malicious patterns, which helps determine the nature of the file. Evaluation results showed high accuracy (98.2%), precision (97.6%), and F1 score (98.2%). However, the study did not extensively address the computational resources required for implementing Markhor across different systems, which is a consideration for practical deployment.

Fawad et al. [35] presented the Multifaceted Deep Generative Adversarial Networks Model (MDGAN) for mobile malware detection. The MDGAN processes hybrid GoogleNet and LSTM features in a pixel-by-pixel pattern through conditional GAN to robustly represent APK files. The generator produces synthetic malicious features, enabling the discriminator to differentiate malware. Experimental validation on the combined AndroZoo and Drebin databases demonstrated a classification accuracy of 96.2% and an F1 score of 94.7%. However, the model was not trained with API call data, limiting its ability to detect malware from Windows API call sequences. Suaboot et al. [36] proposed the Sub-Curve HMM approach for malware detection, focusing on matching subsets of API call sequences from running processes. Hidden Markov Models (HMMs) are trained on these sequences to test the likelihood of matching to the model. Malicious and benign activities are differentiated based on matching scores, which are projected into a curve to detect malicious actions through slope discontinuities. Results from the experiments show that Sub-Curve HMM outperforms existing methods, achieving over 94% accuracy in detecting six malware families, compared to 83% for baseline HMM and 73% for Information Gain. The study highlights improvements in detecting potential data exfiltration incidents but does not address computational resource requirements. Table 1 summarizes the malware detection systems reviewed in this section.

Table 1. Summary of detection techniques used in malware detection using API Calls.

Authors	Algorithm Used	Dataset	Accuracy (%)	Computational Efficiency		
				Training Time (s)	Testing Time (s)	Memory Usage (MB)
Almaleh et al. [2]	Hybrid logistic Regression and RNN	API-Call-Sequences	84.0	0.40	×	×
Cannarile et al. [4]	RF/CatBoost/XGBoost/BiLSTM/BiGRU	API-Call-Sequences/APIMDS	99.3	6.673	0.011	×
Gençaydin et al. [6]	RF/SVM/XGBoost/HGBoost/LSTM	VirusShare/VirusSample	×	×	×	×
Oliveira et al. [11]	DGCNNs/LSTM	API-Call-Sequences	92.44	×	×	×
Catak et al. [14]	LSTM	Mal-API-2019	98.50	2569.2	×	×
Maniriho et al. [29]	CNN/BiGRU	APIMDS/MalbehavD-V1	99.07	35.721	0.177	×
Ki et al. [31]	MSA/LCS	APIMDS	99.80	×	×	×
Suaboot et al. [36]	HMM	VirusShare	99.70	×	×	×
Our Proposed Model	GRU-GAN	APIMDS/API-Call-Sequences/MalbehavD-V1	See Section 4	See Section 4	See Section 4	See Section 4

Several studies [2,4,6,11,29] on malware detection using API calls have utilized various machine learning and deep learning techniques, but few have explored the potential of Generative Adversarial Networks (GANs). GANs can generate realistic synthetic API call sequences, enhancing training data for imbalanced datasets and improving the detection

of sophisticated malware. Despite these advantages, GANs are underutilized due to challenges like high data requirements and training complexity. This paper aimed to overcome these challenges by developing a GAN-based model optimized for efficiency and reduced computational demands. The proposed GAN model demonstrated improved evaluation metrics, including higher accuracy and AUC, lower false-positive rates (FPRs), and reduced false-negative rates (FNRs), owing to its enhanced data generation and advanced learning capabilities.

2.4. Applied Methods

All experiments were performed on a machine running Windows 11 Enterprise edition (64-bit) with an Intel(R) Core(TM) i7-10700 CPU (2.90 GHz), 16 GB RAM, an NVIDIA Quadro P620 graphics card, and a 1 TB hard disk drive. The implementation utilized Python 3.10.15, with TensorFlow 2.3.0 and Keras 2.7.0 as the primary deep learning frameworks, along with additional Python libraries such as Scikit-learn, NumPy, Pandas, Matplotlib, and Seaborn. As illustrated in Figure 1, this paper introduces a hybrid GRU–GAN model for detecting malicious executables based on API call sequences. We evaluated its performance against BiLSTM and BiGRU models using three different datasets. Additionally, we benchmarked these models against existing models discussed in the literature.

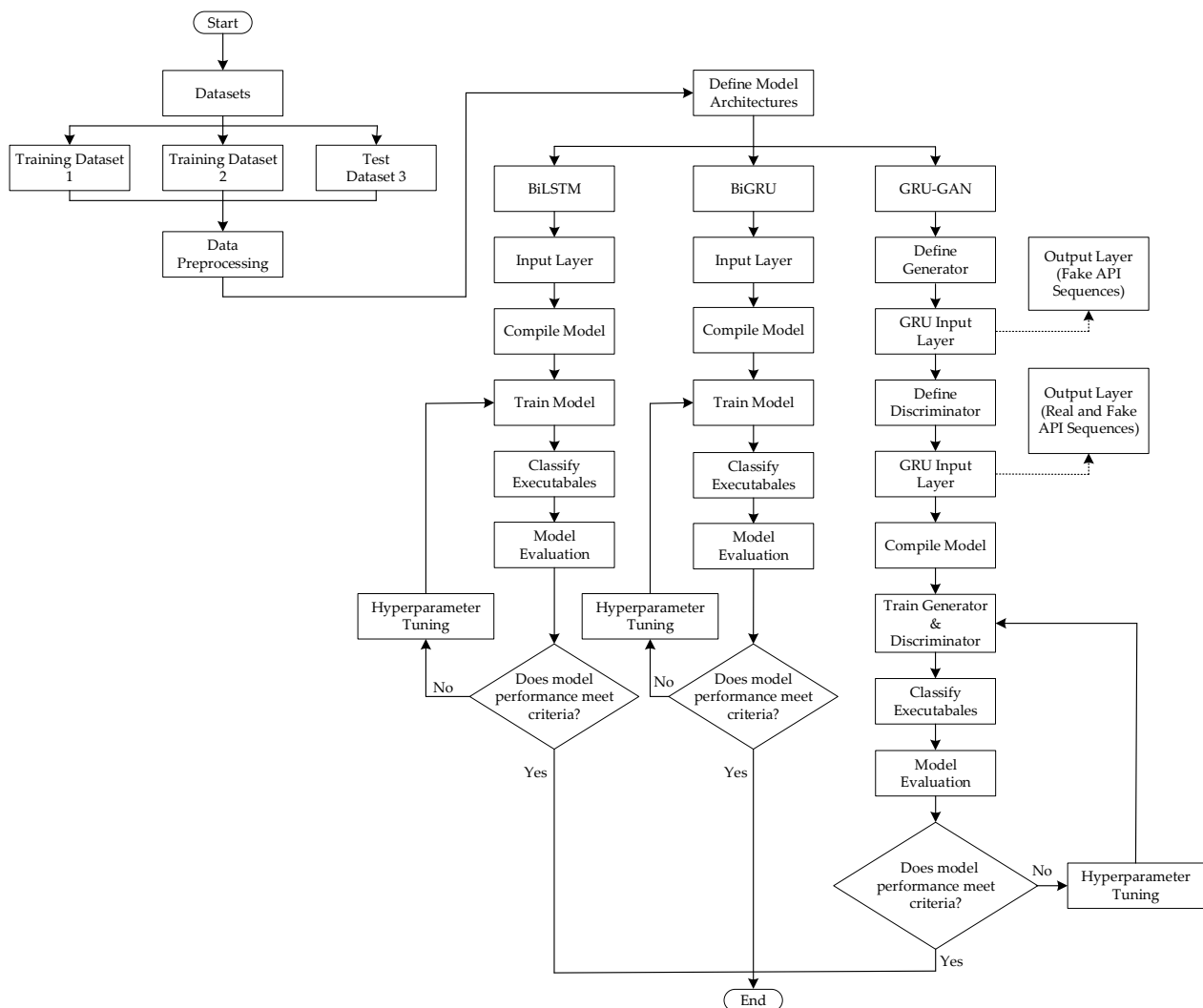


Figure 1. Flowchart of the proposed method.

2.4.1. Datasets

This study utilized three distinct datasets for the training and evaluation of the models, sourced from established repositories in the field of API call sequence analysis. For clarity and consistency, we will refer to these datasets as dataset 1, dataset 2, and dataset 3 throughout this paper, as shown in Figure 1. Dataset 1, originally termed API-Call-Sequences [37], and dataset 2, known as MalbehavD-V1 [38], were used independently for model training, validation, and initial testing. To further assess the robustness and generalizability of the models, we utilized dataset 3, the APIMDS dataset [31], exclusively for additional testing purposes. Dataset 1 [37] consisted of 42,797 malicious and 1079 benign API call sequences, each sequence containing 100 unique, consecutive API calls associated with the parent process extracted from Cuckoo Sandbox reports (features). This dataset exhibited a significant class imbalance, with the malware class outnumbering the benign class by over 40%. Such imbalance can adversely affect model performance, particularly on test data. To address the class imbalance issue, we employed the Synthetic Minority Over-sampling Technique (SMOTE) to balance the classes, resulting in 42,797 samples for each class (malware and benign).

Dataset 2, known as MalbehavD-V1 [38], contains API call sequences from 2570 Windows executable files, evenly split between 1285 benign and 1285 malicious samples. The data, collected through dynamic malware analysis in an isolated Cuckoo sandbox environment, included a total of 153 features. Malware samples, sourced from VirusTotal, were selected from submissions in the second quarter of 2021, ensuring relevance to current threats. Benign samples were downloaded from CNET and scanned for malware using VirusTotal. The dataset was particularly valuable because it represented the behavioral characteristics of current emerging malware types, including ransomware, worms, viruses, spyware, backdoors, adware, keyloggers, and trojans. A comprehensive dataset encompassing diverse malware types is essential for creating robust models capable of accurately detecting and addressing various threats, including emerging ones. By utilizing this dataset, our study benefits from a wide range of malware behaviors, enhancing the ability of the models to identify different types of threats. We utilized dataset 1 (API-Call-Sequences) and dataset 2 (MalbehavD-V1) as our primary training datasets. This selection was based on their comprehensive inclusion of both malware and benign classes, providing a robust foundation for model learning.

Dataset 3, derived from the APIMDS dataset, included API call sequences from 23,080 malware and 300 benign samples, featuring 2727 distinct API calls. The dataset covered diverse malware categories such as backdoors, worms, packed malware, potentially unwanted programs (PUP), trojans, and other types. This variety allowed for a robust evaluation of model performance across different threat categories. We used this dataset exclusively for testing to assess the generalization capabilities and effectiveness of the models in identifying a wide range of malware types, particularly those underrepresented in the training data.

2.4.2. Preprocessing

The preprocessing phase was meticulously applied to all three datasets, with tailored approaches for each based on their specific characteristics. The BiLSTM, BiGRU, and proposed GRU-GAN models were trained and evaluated using all three datasets. For dataset 1, we addressed the class imbalance by implementing the SMOTE technique, which was used to generate synthetic samples of benign API call sequences, preserving their unique characteristics. This approach reduced the risk of overfitting and improves the models' ability to learn from both malware and benign classes equally. Subsequently, the balanced dataset was split into training, validation, and test sets.

Datasets 2 and 3, which primarily contained string features, underwent a more complex preprocessing process. We initiated this process with tokenization, decomposing the API call strings into discrete tokens. This crucial step enabled our deep learning models to effectively interpret and process the structural elements of API calls, enabling the identifi-

cation of patterns and relationships within the API call sequences. Following tokenization, we implemented sequence padding to standardize the length of API call sequences across the datasets. This uniformity in input dimensions is important for efficient batch processing in neural network architectures, optimizing both training and testing processes by allowing the models to handle diverse API calls within a consistent computational framework. The next step involved one-hot encoding vectorization, which transformed the tokenized and padded sequences into numerical representations. This vectorization technique was chosen for its broad compatibility with various deep learning libraries and its adaptability in handling new categories from previously unseen API calls. To further refine the input data, we applied Min-Max normalization, scaling each feature to a specific range from 0 and 1.

For datasets 1 and 2, we performed a percentage split: 70% for model training, 15% for validation, and 15% for testing on each dataset. This stratification ensured a robust evaluation approach. Notably, we further enhanced the rigor of our evaluation process by utilizing dataset 3 as an additional test set. The resulting preprocessed data served as the input data for training and evaluating the various models discussed in subsequent sections of this paper.

2.4.3. The Deep Learning Architectures

In this subsection, we explore the deep learning architectures employed in our study, focusing on Bidirectional Long Short-Term Memory (BiLSTM), Gated Recurrent Unit (GRU), Bidirectional Gated Recurrent Unit (BiGRU), and Generative Adversarial Networks (GANs). These architectures are pivotal for modeling sequential data and generating synthetic samples, enhancing our method of malware detection.

- Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are deep learning algorithms ideal for modeling sequential data, such as time-series and natural language processing tasks [39]. RNNs maintain an internal state to capture information from previous inputs, but traditional RNNs struggle with long sequences due to the vanishing gradient problem. Long Short-Term Memory (LSTM) networks address this issue with memory cells regulated by gates, while Gated Recurrent Units (GRUs) simplify LSTM architecture with faster training and comparable performance [40].

On the other hand, Bidirectional LSTMs (BiLSTMs) and Bidirectional GRUs (BiGRUs) enhance sequence modeling by processing data in both forward and backward directions, capturing comprehensive patterns [41]. BiLSTMs are particularly effective in applications like natural language processing and time-series analysis due to their ability to consider both past and future inputs [42]. These advancements have significantly improved the handling of long-term dependencies, enhancing performance in several applications, including speech recognition, machine translation, sentiment analysis, and API call analysis.

- Generative Adversarial Network

Generative Adversarial Networks (GANs) are a cutting-edge approach in deep learning, leveraging two neural networks consisting of a generator (G) and a discriminator (D) [43]. These networks are trained concurrently through adversarial processes. In the context of malware detection, GANs demonstrate exceptional performance by learning the distributions of benign and malicious API call sequences. The generator produces synthetic sequences that mimic real ones, while the discriminator is tasked with distinguishing between these synthetic sequences and realistic API call sequences. This adversarial training refines the model, capturing hidden details in API behaviors that other methods might miss, enabling the detection of sophisticated malware. GANs address imbalanced datasets by generating synthetic malware sequences enhancing training data and detection performance. They also improve the robustness of existing systems against adversarial attacks and evasion techniques.

Furthermore, GANs aid in identifying malware variants and families, contributing to threat intelligence [44]. While GANs offer significant potential, their application to malware

detection using API calls remains relatively unexplored. Several factors contribute to this limited adoption, including substantial data requirements, complex training processes, and challenges in model interpretability. We addressed these issues by optimizing the GAN architecture through hyperparameter tuning and strategically incorporating GRU units. This approach enabled us to fully leverage the capabilities of GANs while mitigating their inherent limitations in the context of malware detection.

We followed a structured implementation of the Bidirectional Long Short-Term Memory (BiLSTM) and Bidirectional Gated Recurrent Unit (BiGRU) models, as depicted in Figure 1. Both architectures leveraged bidirectional processing to capture patterns in API call behaviors, enabling a detailed understanding of both past and future contextual information within the sequences. Each model employed a sequential architecture, with an input layer designed to receive preprocessed API call data. The BiLSTM model incorporated a bidirectional LSTM layer, while the BiGRU model utilized a bidirectional GRU layer. Dense layers were then used for feature extraction and classification. The final layer in both architectures was a dense layer with a sigmoid activation function, enabling binary classification of executables as either malicious or benign. Both models were compiled using binary cross-entropy as the loss function and the Adam optimizer. The training process for both models used the preprocessed data from training datasets 1 and 2. Data were processed in batches over multiple epochs, with a portion reserved for validation to monitor overfitting. Post-training, the models classified executables based on their API call sequences, applying a threshold to the output probabilities to determine the final classification. Evaluation of the BiLSTM and BiGRU models encompassed a comprehensive set of metrics, including accuracy, precision, recall, F1 score, and ROC-AUC.

In cases where the initial performance fell short of predetermined criteria, we engaged in an iterative hyperparameter tuning process. This method involved adjusting key parameters such as learning rate, layer units, dropout rates, and batch size, employing random search techniques to optimize model configurations. A summary of the optimal hyperparameter settings used for the various models is provided in Table 2. After meeting performance criteria on the validation and test sets, both models were subjected to a final evaluation using dataset 3, as previously discussed.

Table 2. A summary of the different parameter settings for the models.

Algorithms	Parameters Used	Values
BiLSTM and BiGRU	Input_dim	Vocab_size
	Output_dim	128
	Input_length	Max_length
	Learning rate	0.001
	Dense Unit	64
	Activation Input	relu
	Activation Output	sigmoid
	Optimizer	Adam
GRU-GAN	Epoch	50
	Learning rate	0.001
	Dense Layers unit	258, 128, 64
	Activation Layer Input	Relu
	Activation layer output	Sigmoid
	Kernel_regularizer	11, 12
	BatchNormalization Layer	Sequence_length
	Dropout Layer	0.5
	Manual Seed	2

2.4.4. The Hybrid Model

The implementation of the GRU-GAN (Gated Recurrent Unit–Generative Adversarial Network) model is depicted in Figure 2. This architecture combines the sequential processing strengths of GRUs with the adversarial training features of GANs, providing a

unique solution to the complexities of malware detection. The process begins with data preparation, utilizing three distinct datasets: training data (dataset 1 and dataset 2) and test data (dataset 3). The preprocessing stage extracts API call sequences from these executable files, forming the basis for model training and evaluation.

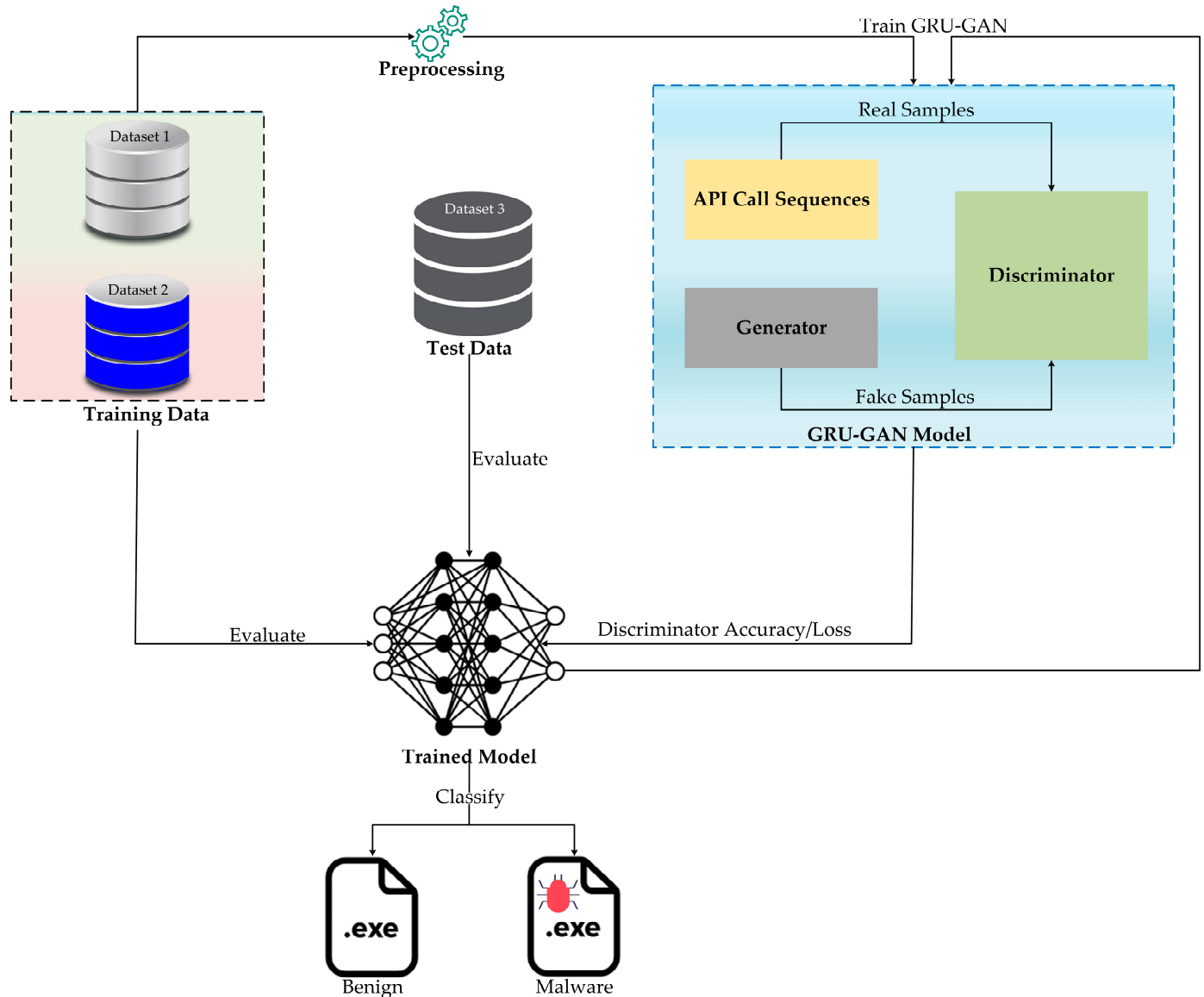


Figure 2. The architecture of the proposed GRU-GAN model.

The GRU-GAN architecture consists of two primary components: a generator with four GRU layers (258, 128, and 64 hidden units) that produces fake API call sequences and a discriminator with two GRU layers (64 and 100 hidden units) that distinguishes between real and fake sequences. The training process involves meticulous hyperparameter tuning to optimize learning rates, GRU unit numbers, and noise dimensions, followed by adversarial training where the generator and discriminator engage in alternating cycles of improvement. This phase leverages the training datasets to refine the model's ability to generate realistic malware-like sequences and accurately differentiate between benign and malicious patterns.

Post-training, the model's effectiveness was rigorously evaluated using the test data (dataset 3), measuring key metrics such as discriminator accuracy, precision, recall, F1 score, AUC-ROC, false-positive rate, and false-negative rate. For real-world applications, new executable files undergo API call sequence extraction before being classified by the

trained discriminator. The performance of the model is continuously monitored, with ongoing fine-tuning based on accuracy metrics and emerging malware samples, ensuring its adaptability to evolving threats. This comprehensive approach, combining the sequential processing capabilities of GRUs with the adversarial learning paradigm of GANs, provides an enhanced solution for detecting increasingly complex malware behaviors. For a clearer understanding of the implementation process, we present Algorithm 1 of the proposed hybrid model.

Algorithm 1. Malware detection models.	
Input:	S, L, E, M
Output:	P, R
1.	trainData, testData = splitData($S, L, 0.8$)
2.	trainSeq, testSeq = padSequences(trainData, M), padSequences(testData, M)
3.	$V = \text{uniqueAPICallsCount}(S)$
4.	Define and train models: biLSTM, biGRU, gruGAN
5.	$G = \text{Generator}$ and $D = \text{Discriminator}$
6.	Train biLSTM, biGRU on trainSeq
7.	for epoch = 1 to 50 do
8.	fakeSamples = $G.\text{generate}(\text{batchSize})$
9.	$D.\text{train}(\text{realSamples}, \text{fakeSamples})$
10.	$G.\text{train}(\text{through} = \text{gruGAN})$
11.	end for
12.	Ensemble prediction:
13.	$\text{Pred1}, \text{Pred2}, \text{Pred3} = \text{biLSTM.predict}(\text{testSeq}), \text{biGRU.predict}(\text{testSeq}), \text{gruGAN.predict}(\text{testSeq})$
14.	$P = \text{average}(\text{Pred1}, \text{Pred2}, \text{Pred3})$
15.	Evaluate Performance:
16.	$R = \text{calculateMetrics}(P, \text{tetsLabels})$
17.	return P, R
18.	end

3. Results

This section outlines the experimental evaluations conducted to assess the performance of the models, emphasizing the outstanding performance of the proposed GRU–GAN model. The results were derived from a binary classification task aimed at differentiating between malware and benign executable files within the Windows operating system.

3.1. Performance Evaluation

In this subsection, we provide insights into the effectiveness and efficiency of the models based on the three datasets.

3.1.1. Results of BiLSTM, BiGRU, and GRU–GAN Models during Training on Dataset 1 and Dataset 2

The training performance of the BiLSTM, BiGRU, and GRU–GAN models on dataset 1 is depicted in Figures 3 and 4. Both the BiLSTM and BiGRU models demonstrated rapid convergence during the initial training epochs, achieving over 90% accuracy for both training and validation sets by epoch 5. While BiGRU exhibited slightly more volatile training accuracy, both models ultimately converged to comparable final accuracies of approximately 96–97%. The corresponding loss curves for both models showed a similar pattern of rapid initial decrease followed by gradual stabilization at low levels. These results show that both BiLSTM and BiGRU are effective models for the task of malware classification.

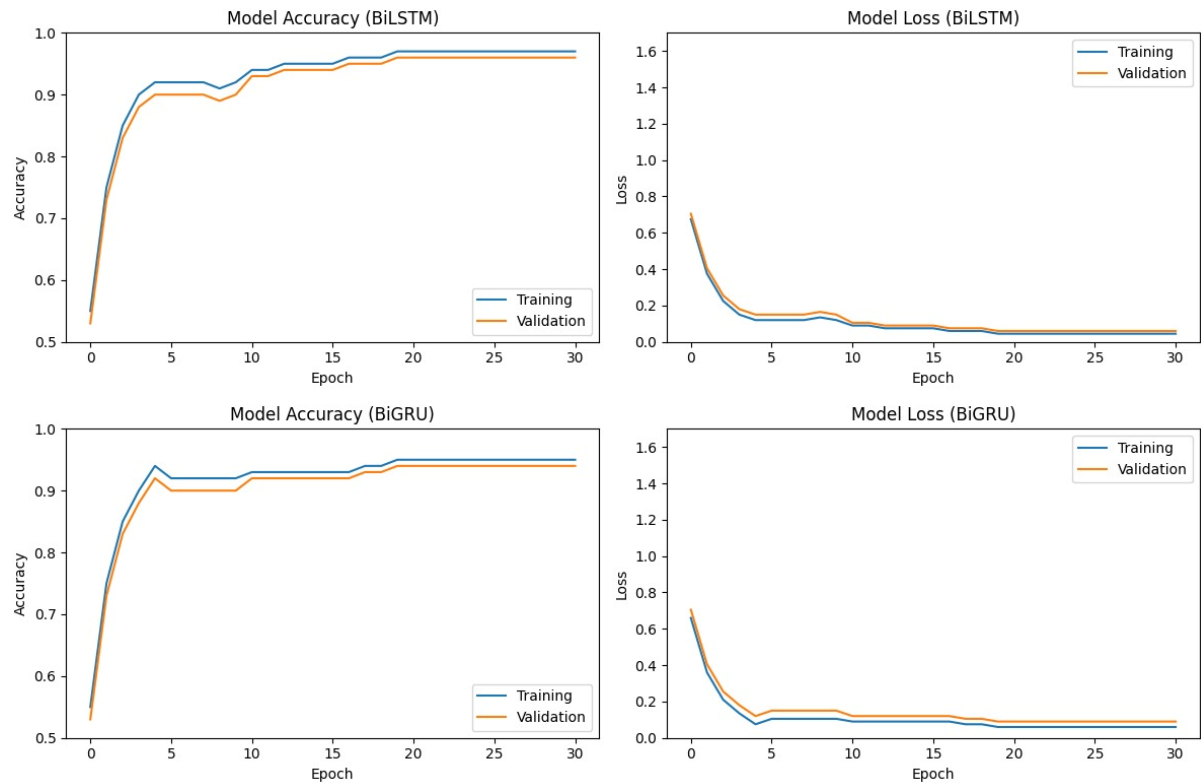


Figure 3. Accuracy and validation loss of the BiLSTM and BiGRU models on dataset 1.

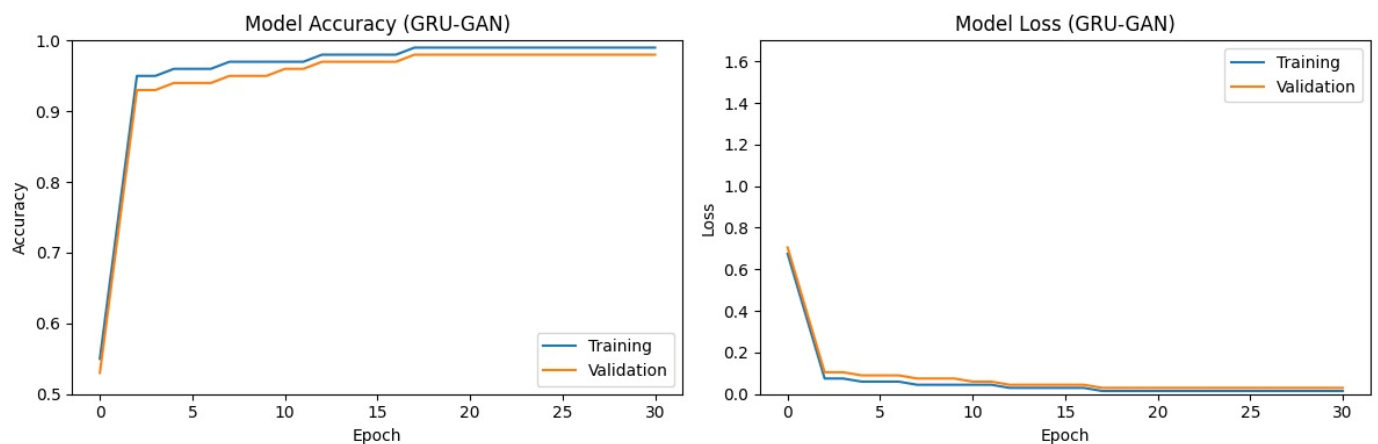


Figure 4. Discriminator accuracy and validation loss of the GRU-GAN model on dataset 1.

On the other hand, the GRU-GAN model exhibited a more rapid initial learning curve, surpassing 95% accuracy by epoch 5. This learning ability captured discriminative features faster compared to the BiLSTM and BiGRU models. While maintaining a slight accuracy advantage throughout training, the GRU-GAN ultimately converged to final accuracies of approximately 98–99%. While achieving higher classification accuracy, the GRU-GAN exhibited a marginally higher final loss, potentially due to a lack of diversity in the generated data.

Figures 5 and 6 illustrate the training performance of the BiLSTM, BiGRU, and GRU-GAN models on dataset 2. Both the BiLSTM and BiGRU models demonstrated comparable performance, with rapid initial accuracy gains and subsequent gradual improvement. The training accuracy for both models consistently outperformed the validation accuracy by a small margin. Final accuracies of approximately 95–97% (training) and 94–96% (validation) were achieved after 30 epochs. The corresponding loss curves illustrate this

pattern, showing a sharp initial decrease followed by a more gradual decline. The close convergence of training and validation losses depicts minimal overfitting of the models.

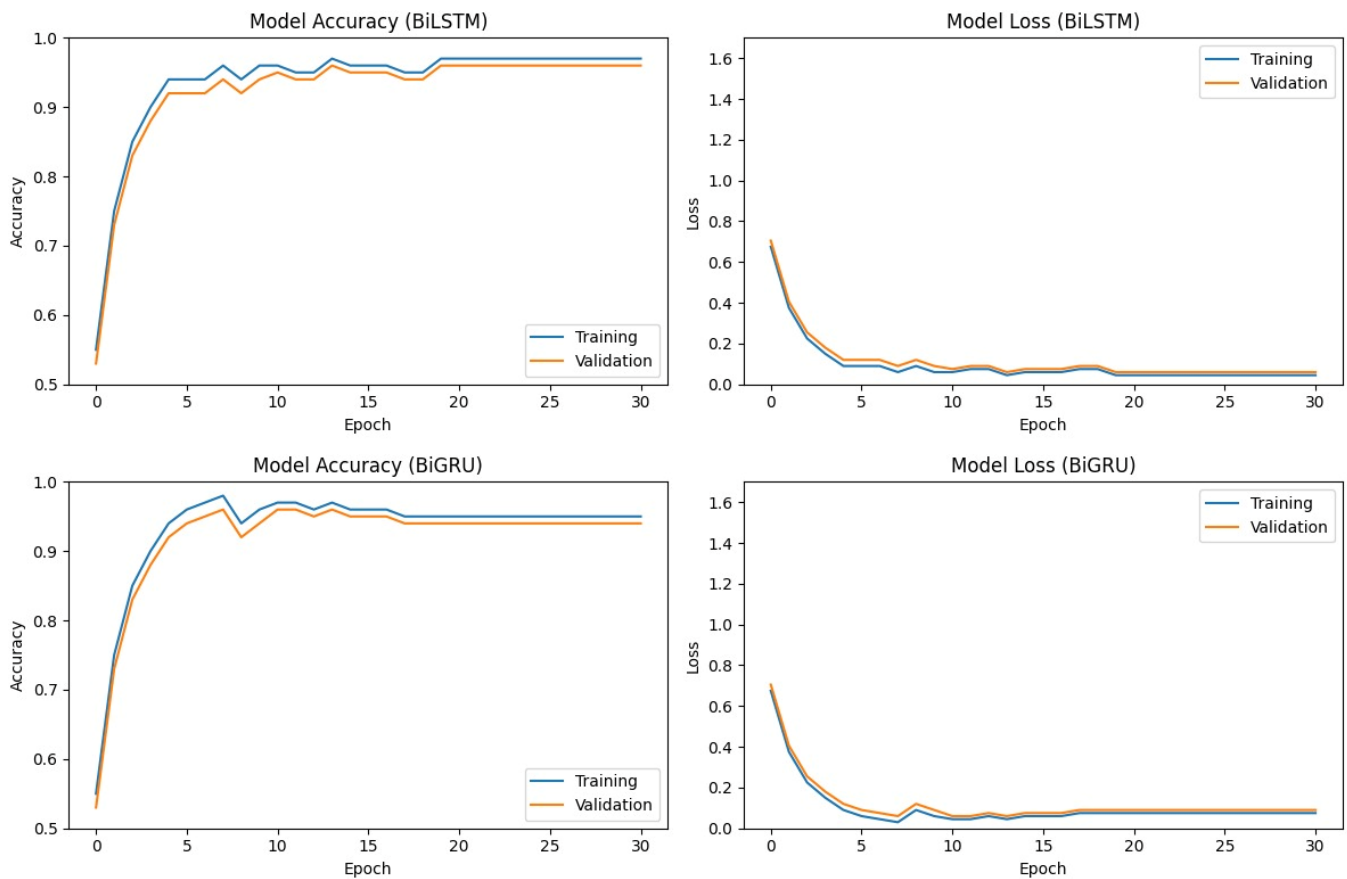


Figure 5. Accuracy and validation loss of the BiLSTM and BiGRU models on dataset 2.

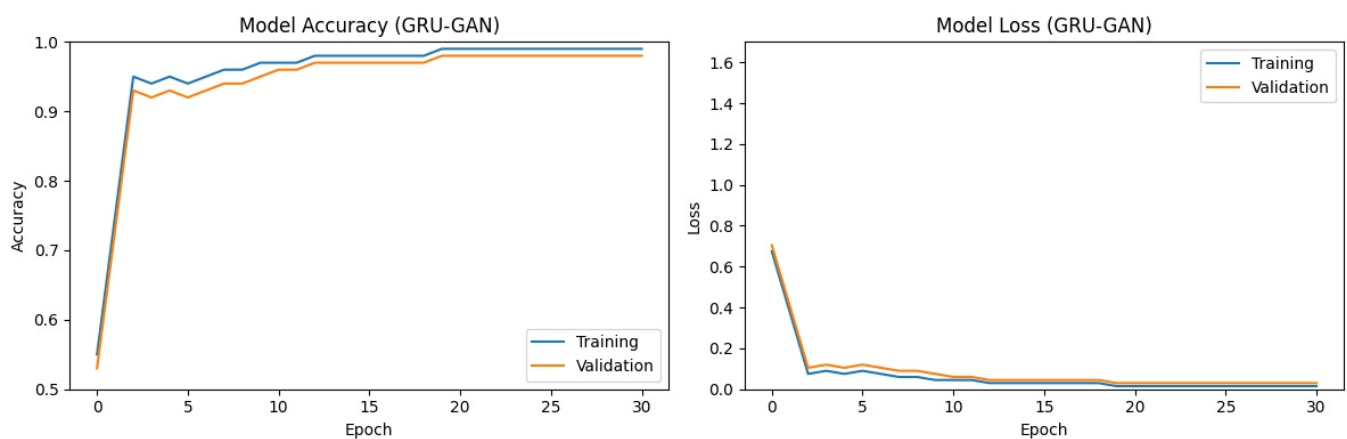


Figure 6. Discriminator accuracy and validation loss of the GRU-GAN model on dataset 2.

The GRU-GAN model exhibited a distinctive learning pattern characterized by a rapid initial increase in accuracy and sustained training–validation gap. While achieving near-perfect training accuracy, the validation accuracy plateaued around 97–98%. The corresponding loss curve demonstrated a steep initial decline followed by a more gradual decrease. While all three models performed well in classifying API call sequences, the GRU-GAN model showed slightly superior final accuracy.

3.1.2. Performance Metrics of BiLSTM, BiGRU, and GRU-GAN Models on Dataset 1, Dataset 2, and Dataset 3

On dataset 1, shown in Figure 7A, the BiLSTM model showed a relatively high number of TN (correctly identified benign files) and TP (correctly identified malware) but also exhibited some FP and FN. For instance, it correctly identified 3986 benign files (TN) and 5595 malware files (TP) but misclassified around 242 benign files as malware (FP) and 208 malware files as benign (FN). The BiGRU model demonstrated similar performance, with lower FP but higher FN rates. In contrast, the GRU-GAN model demonstrated near-perfect classification, with TN and TP values of 4228 and 5798, respectively. Notably, the GRU-GAN model did not misclassify benign samples as malware (FP) but misclassified five malware samples as benign.

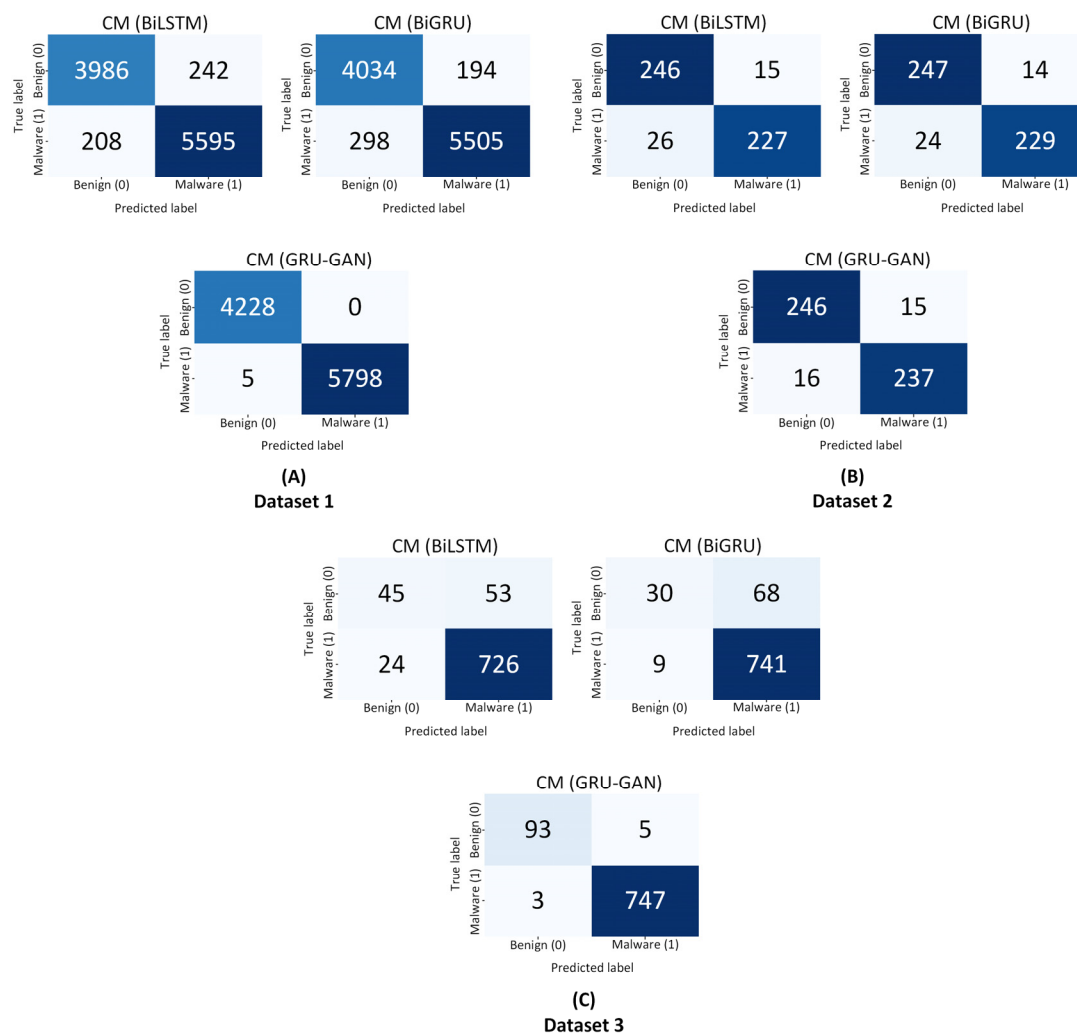


Figure 7. Confusion matrix of the BiLSTM, BiGRU, and GRU-GAN models on datasets 1, 2 and 3.

Figure 7B, representing dataset 2, shows a slight decline in performance for BiLSTM and BiGRU. The BiLSTM model, for example, correctly identified 246 benign files (TN) and 227 malware files (TP) but misclassified approximately 15 benign files as malware (FP) and 26 malware files as benign (FN). The BiGRU model showed similar trends. The GRU-GAN model, however, maintained its high performance, with TN and TP values of 246 and 237 each and relatively low FP and FN rates.

Dataset 3 in Figure 7C presents the most challenging scenario for all models. The BiLSTM and BiGRU models showed a significant increase in FP rates, with BiLSTM misclassifying 53 benign files as malware (FP) while correctly identifying only about 45 benign

files (TN). This high FP rate could lead to numerous false alarms in a real-world setting. Remarkably, the GRU–GAN model maintained its high accuracy even on this challenging dataset, with TN and TP values of 93 and 747, respectively, and FP and FN rates staying minimal. The consistently high TN and TP rates of the GRU–GAN model across all datasets, coupled with its minimal FP and FN rates, underscore its superior performance in malware detection. This performance stability is required in detection applications, where false positives can lead to unnecessary alerts and resource allocation, while false negatives could allow malicious programs to go undetected. The BiLSTM and BiGRU models' increased FP rates on the more challenging datasets suggest they may be overly sensitive, potentially flagging benign software as malicious. Such outcomes could lead to inefficiencies and false alarms in real-world applications.

Furthermore, the evaluation metrics in Table 3 reveal that the GRU–GAN model outperformed BiLSTM and BiGRU on imbalanced data, achieving an accuracy of 98% and an AUC of 98% with low error rates. Conversely, BiLSTM and BiGRU showed decreased performance with accuracy of around 94% and 92%, respectively.

Table 3. Evaluation metrics results of the BiLSTM, BiGRU, and GRU–GAN models on the original (imbalance) dataset 1.

	Accuracy	Precision	Recall	F1 Score	AUC	FPR	FNR
BiLSTM	0.941	0.936	0.945	0.943	0.89	0.123	0.071
BiGRU	0.925	0.914	0.925	0.924	0.87	0.141	0.093
GRU–GAN	0.985	0.974	0.984	0.985	0.98	0.005	0.007

Table 4 provides a detailed comparative analysis of the BiLSTM, BiGRU, and GRU–GAN models across three datasets. On dataset 1, all models performed well, with BiLSTM achieving an accuracy of 95%, BiGRU achieving 94%, and GRU–GAN achieving nearly perfect scores across all metrics, including an AUC of 99%. Dataset 2 presents more challenges, with BiLSTM and BiGRU showing decreased performance but GRU–GAN maintaining a high accuracy of 98% and precision of 99%. On dataset 3, the most difficult, BiLSTM and BiGRU's performance dropped significantly, with accuracies falling to 90% and 80%, respectively, while GRU–GAN achieved an accuracy of 99%. Figure 8 visually shows these results, highlighting GRU–GAN's consistent superiority across all datasets.

Table 4. Evaluation metrics results of the BiLSTM, BiGRU, and GRU–GAN models on the three datasets.

Dataset 1							
	Accuracy	Precision	Recall	F1 score	AUC	FPR	FNR
BiLSTM	0.958	0.962	0.966	0.964	0.98	0.057	0.035
BiGRU	0.947	0.958	0.954	0.956	0.98	0.045	0.051
GRU–GAN	0.999	0.999	0.999	0.999	0.99	0.000	0.001
Dataset 2							
BiLSTM	0.944	0.942	0.942	0.944	0.97	0.057	0.102
BiGRU	0.924	0.921	0.923	0.923	0.97	0.053	0.094
GRU–GAN	0.981	0.997	0.998	0.992	0.98	0.057	0.063
Dataset 3							
BiLSTM	0.903	0.775	0.734	0.747	0.82	0.540	0.032
BiGRU	0.805	0.716	0.714	0.756	0.79	0.693	0.012
GRU–GAN	0.998	0.997	0.996	0.997	0.99	0.051	0.004

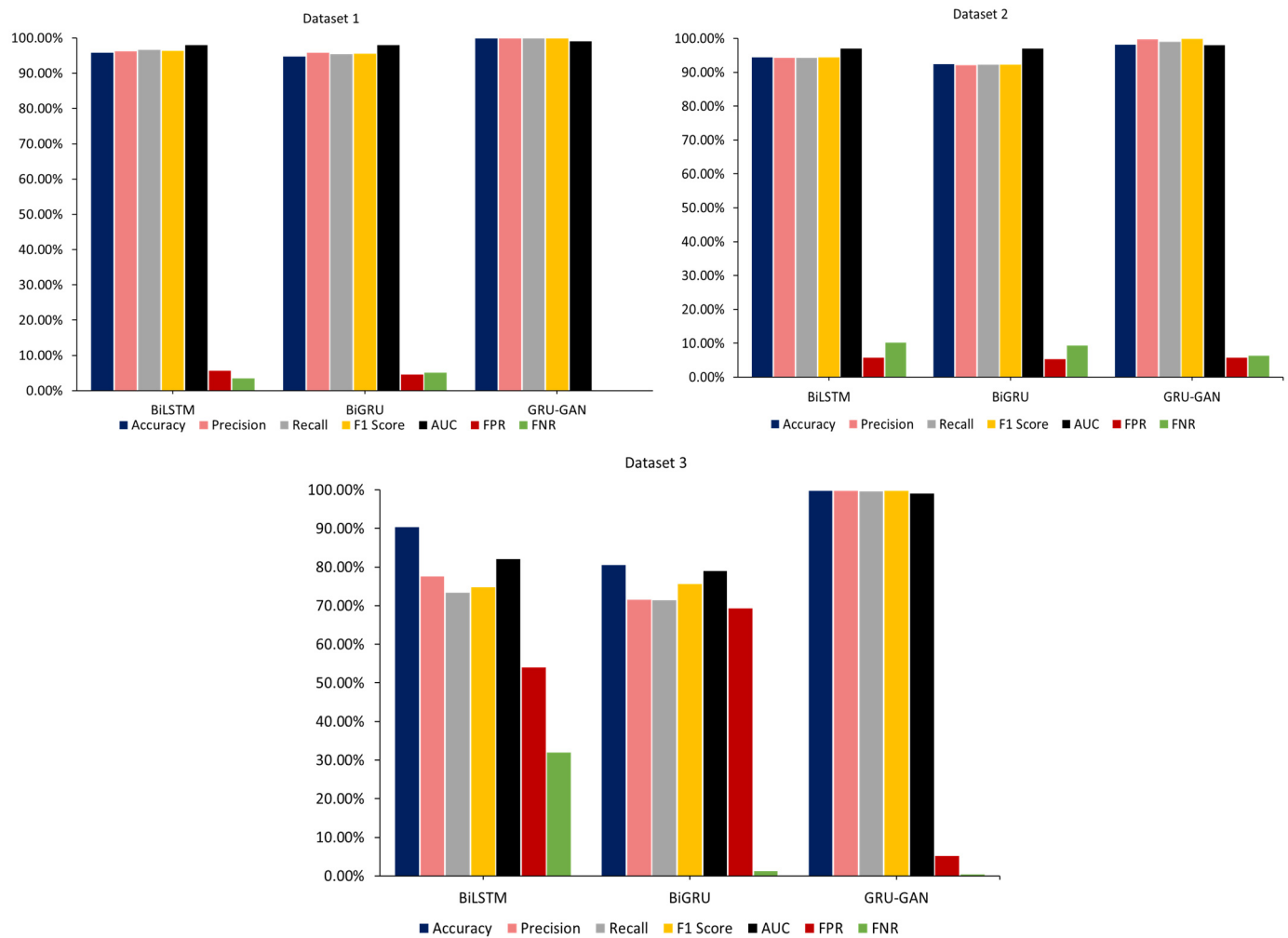


Figure 8. Evaluation metrics results of the BiLSTM, BiGRU, and GRU-GAN models on the three datasets.

The Receiver Operating Characteristic (ROC) curves in Figure 9 visually depict the performance of BiLSTM, BiGRU, and GRU-GAN across the three datasets. On dataset 1, all models exhibited strong performance, with BiLSTM and BiGRU achieving AUC values of 98%. GRU-GAN surpassed both, reaching an AUC of 99%, demonstrating a clear distinction between malware and benign samples. Dataset 2 proved more challenging, with BiLSTM and BiGRU achieving AUC values of 97%. GRU-GAN maintained superior performance with an AUC of 98%, showcasing robustness and effective generalization to complex data. Dataset 3, the most challenging, saw BiLSTM and BiGRU decline to AUC values of 82% and 79%, respectively.

On the other hand, the GRU-GAN model maintained an exceptional AUC of 99%, underscoring its ability to handle complexity and variability in API call sequences. The consistently high performance of the GRU-GAN model across all datasets and maintaining remarkable AUC scores highlights its superior capability in classifying unseen executables.

In addition, Figure 10 illustrates the prediction performance on dataset 1. All three models correctly classified a significant portion of both malicious and benign samples, though with visible misclassifications. However, the GRU-GAN model stands out with its excellent classification. It showed an ideal separation between malicious and benign samples. The BiLSTM and BiGRU models, while performing well, exhibited some overlap between the two classes, indicating a higher rate of misclassification compared to GRU-GAN.

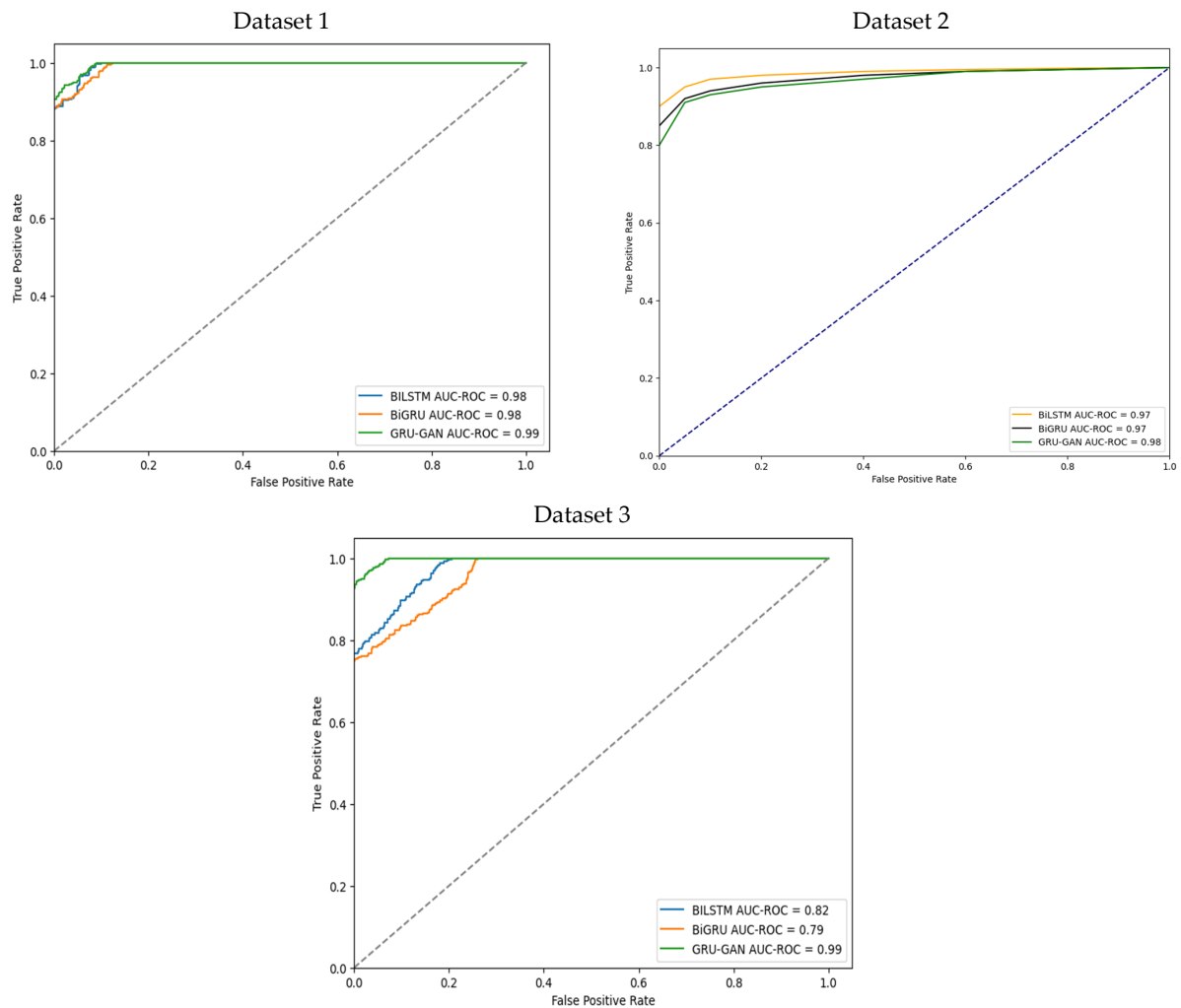


Figure 9. ROC curve results of the BiLSTM, BiGRU, and GRU-GAN models on the three datasets.

Figure 11, representing dataset 2 with fewer test samples, reveals a slight degradation in performance for the BiLSTM and BiGRU models. These models showed an increased overlap between the malicious and benign classifications, suggesting a higher rate of false positives and false negatives.

In comparison, the GRU-GAN model maintained its high performance, continuing to show clear separation between the two classes with minimal misclassifications. Figure 12, depicting results from dataset 3, highlights the most significant differences among the models. The BiLSTM and BiGRU models struggled with this dataset, showing substantial overlap between malicious and benign classifications. This indicates a high rate of misclassification, particularly false positives, where benign programs were incorrectly flagged as malicious. Such performance could lead to numerous false alarms in a real-world setting. Significantly, the GRU-GAN model maintained its high accuracy even on this challenging dataset. It continued to show a clear separation between malicious and benign samples, with only a minimal increase in misclassifications compared to its performance on the previous datasets. The consistent superior performance of the GRU-GAN model across all three datasets is particularly noteworthy. Its ability to maintain a clear separation between malicious and benign samples, even in the most challenging dataset, indicates its effectiveness in detecting hidden patterns in API call sequences that distinguish malware from benign executables. This robust performance reveals that the GRU-GAN model has learned more generalized and discriminative features compared to the BiLSTM and BiGRU models.

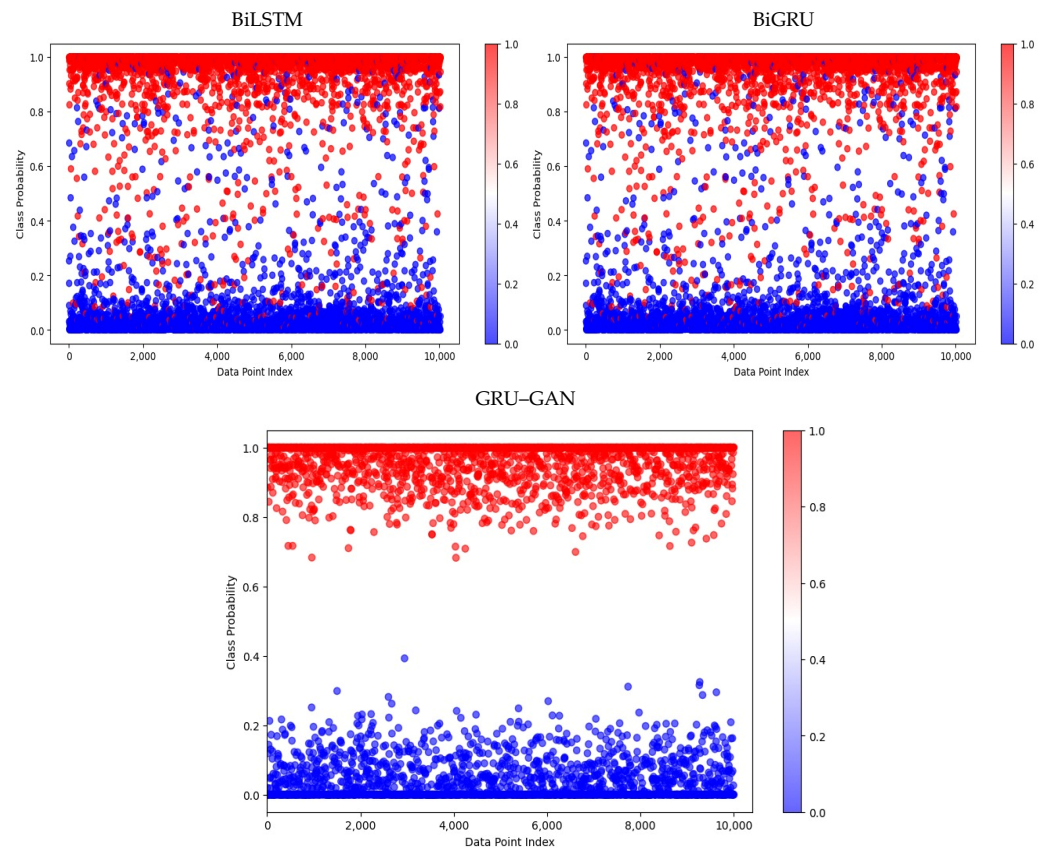


Figure 10. Prediction performance of the BiLSTM, BiGRU, and GRU-GAN models on dataset 1.

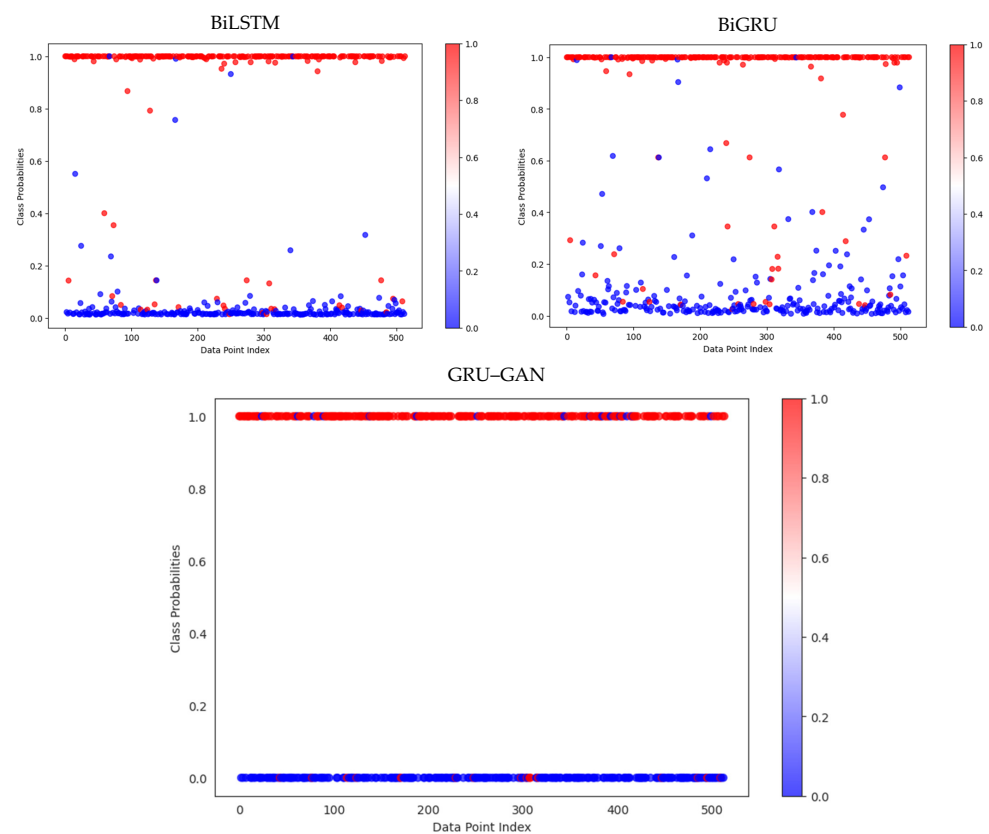


Figure 11. Prediction performance of the BiLSTM, BiGRU, and GRU-GAN models on dataset 2.

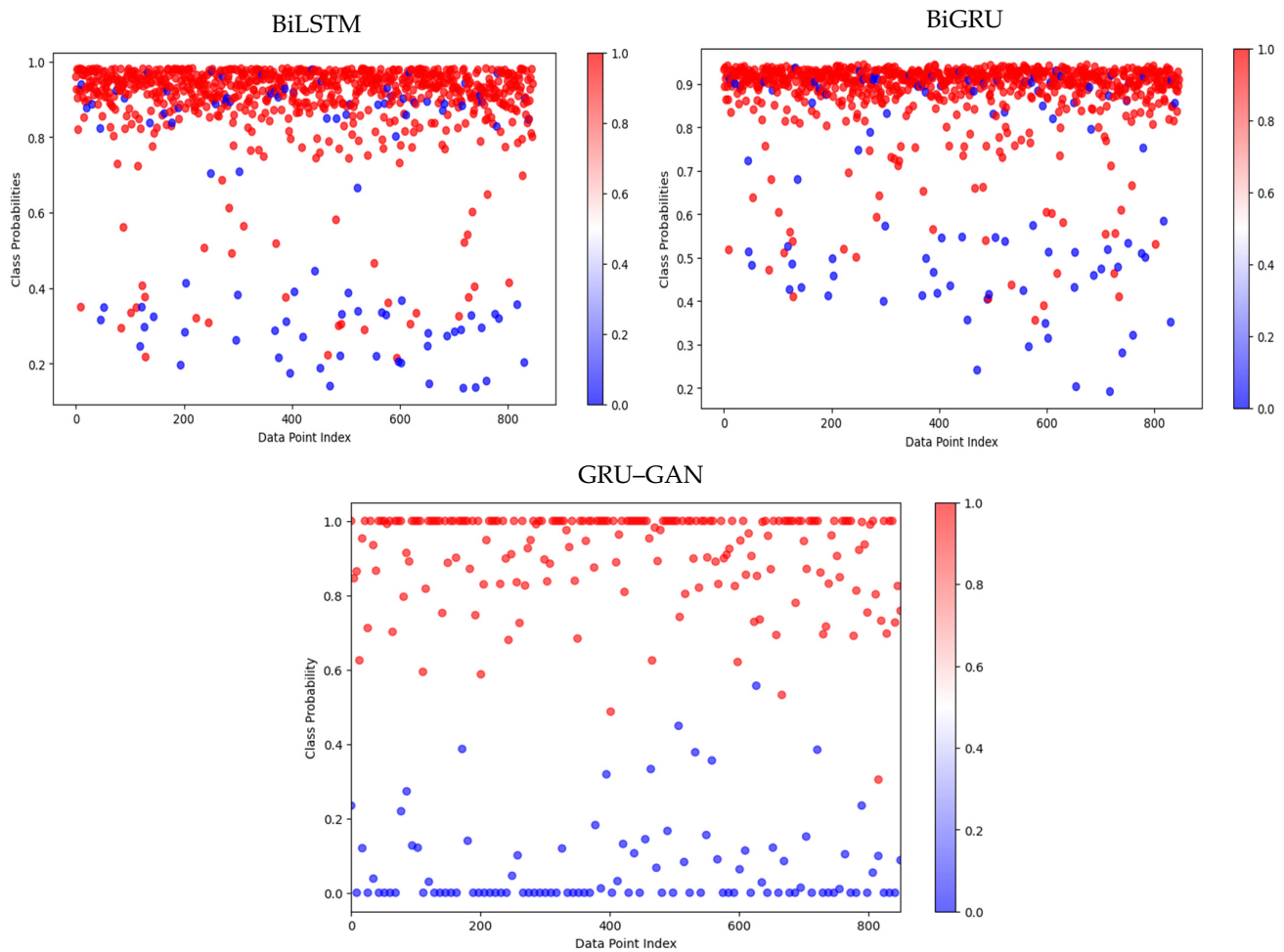


Figure 12. Prediction performance of the BiLSTM, BiGRU, and GRU-GAN models on dataset 3.

3.1.3. The Computational Complexity of BiLSTM, BiGRU, and GRU-GAN Models

The computational efficiency of deep learning models is a key factor in their practical application, particularly in malware detection, where real-time performance is often crucial. Table 5 and Figure 13 provide a detailed overview of the computational resources utilized by the BiLSTM, BiGRU, and GRU-GAN models on datasets 1 and 2, offering valuable insights into their efficiency in terms of training time, testing time, and memory usage. We have intentionally excluded dataset 3 from this computational evaluation, as it was utilized exclusively for testing the performance of the models rather than for training or comprehensive resource analysis. On dataset 1, the BiLSTM model demonstrated moderate computational requirements, with a training time of 80 s, a testing time of 12.7 s, and a memory usage of 120.3 MB. The BiGRU model showed improved efficiency, requiring 69 s for training, 8.4 s for testing, and 90 MB of memory. The GRU-GAN model, on the other hand, outperformed both in terms of computational efficiency, completing training in 57 s, testing in 7.2 s, and utilizing only 75 MB of memory. This performance indicates that despite its more complex architecture, the GRU-GAN model is more computationally efficient than the other models evaluated. Dataset 2, being potentially more complex, requires increased computational resources across all models. The BiLSTM model's training time increased to 127 s, with a testing time of 14.8 s and memory usage of 147 MB. The BiGRU model showed a similar trend, requiring 112 s for training, 11.5 s for testing, and 104 MB of memory. However, the GRU-GAN model demonstrated superior efficiency, completing training in 98 s, testing in 9.2 s, and utilizing 90 MB of memory. This consistent performance across datasets highlights the scalability and efficiency capabilities of the GRU-GAN model in handling larger and more complex datasets.

Table 5. Computation resources used by the BiLSTM, BiGRU, and GRU–GAN models.

Dataset 1			
	Training Time (s)	Testing Time (s)	Memory Usage (MB)
BiLSTM	80	12.7	120.3
BiGRU	69	8.4	90
GRU–GAN	57	7.2	75
Dataset 2			
BiLSTM	127	14.8	147
BiGRU	112	11.5	104
GRU–GAN	98	9.2	90

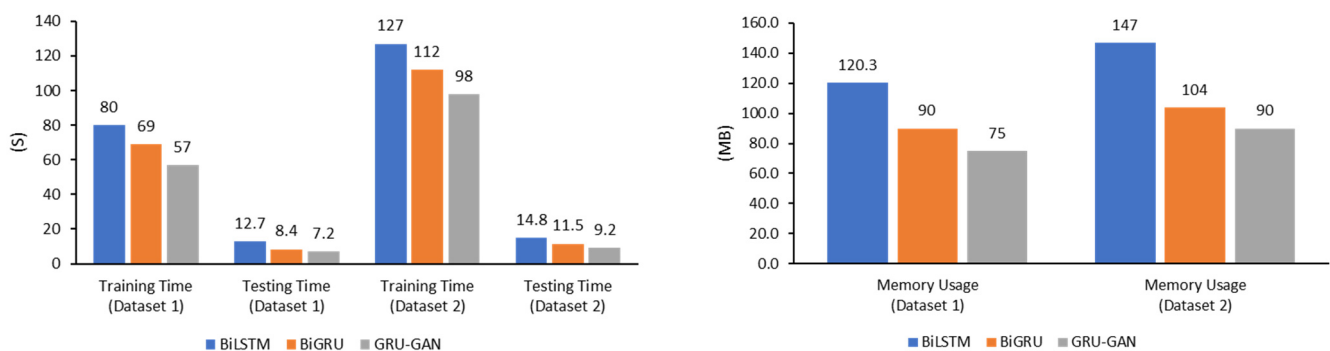
**Figure 13.** Computation resources used by the BiLSTM, BiGRU, and GRU–GAN models.

Figure 13 visually supports these findings, clearly illustrating the computational advantage of the GRU–GAN model across both datasets. The graph demonstrates that the GRU–GAN consistently required less training time, testing time, and memory compared to both the BiLSTM and BiGRU models. This visual representation underscores the scalability of the GRU–GAN model, as it maintained its computational edge even as the dataset complexity increased. Conventional reasoning might suggest that the additional complexity of the GAN framework would lead to increased computational demands. However, the results indicate that the GRU–GAN model not only achieved higher accuracy, as discussed in previous sections, but also did so with lower computational overhead. This efficiency can be attributed to the ability of GAN to learn more effective representations of the data, requiring fewer iterations to achieve optimal performance.

The reduced training and testing times of the GRU–GAN model have significant implications for real-world applications in malware detection. Faster training times allow for more frequent model updates, enabling the system to adapt quickly to new malware variants. The shorter testing times are important for real-time threat detection, where rapid identification of potential malware is essential to prevent security breaches. Moreover, the lower memory usage of the GRU–GAN model is advantageous for deployment in resource-constrained environments or for integration into existing security systems without requiring significant hardware upgrades. This efficiency could facilitate the wider adoption of advanced deep learning-based malware detection systems across various computing platforms.

3.1.4. Comparison with Existing Models

As shown in Table 6, we compared the GRU–GAN model with existing malware detection models, focusing on key metrics such as accuracy, training time, testing time, and memory usage. Our proposed GRU–GAN model demonstrated exceptional performance, achieving an accuracy of 99.99%, surpassing models by Almaleh et al. [2], Maniriho

et al. [29], Cannarile et al. [4], Catak et al. [14], and Ki et al. [31]. In terms of computational efficiency, our model was highly efficient, with a training time of 57 s, significantly faster than the model proposed by Catak et al. [14] at 2569.2 s and Maniriho et al. [29] at 35.721 s. Only the model proposed by Almaleh et al. [2] was faster at 0.40 s but with lower accuracy (84.0%). The GRU–GAN model achieved a testing time of 7.2 s, which is competitive, although slower than Cannarile et al. [4] at 0.011 s and Maniriho et al. [29] at 0.177 s. It achieved this while maintaining higher accuracy, indicating a balanced solution for real-world applications where both speed and accuracy are critical. The GRU–GAN model used 75 MB of memory efficiently. Many compared studies did not report memory usage, highlighting a gap in evaluation that our study addresses thoroughly.

Table 6. Comparison of the proposed model with existing research.

Authors	Accuracy (%)	Computational Efficiency		
		Training Time (s)	Testing Time (s)	Memory Usage (MB)
Almaleh et al. (2023) [2]	84.0	0.40	×	×
Maniriho et al. (2023) [29]	99.07	35.721	0.177	×
Cannarile et al. (2022) [4]	99.3	6.673	0.011	×
Catak et al. (2020) [14]	98.50	2569.2	×	×
Ki et al. (2015) [31]	99.80	×	×	×
Our Proposed Model	99.99	57	7.2	75

4. Discussion

The results presented in this paper demonstrate the effectiveness of the proposed GRU–GAN model. The model consistently outperformed the BiLSTM and BiGRU models across multiple datasets, highlighting its robustness and generalizability. Several factors contribute to the superior performance of the GRU–GAN model. First, the adversarial training process of GANs enables the model to learn more robust and generalizable features from API call sequences, allowing it to capture subtle patterns that distinguish malicious from benign behavior, even in complex or previously unseen samples. Second, the integration of GRUs enhances the model’s ability to process sequential data, which is crucial for effectively analyzing API call sequences.

In addition to achieving higher accuracy, the GRU–GAN model demonstrated improved computational efficiency compared to the BiLSTM and BiGRU models. Across the datasets, the GRU–GAN consistently required less training time, testing time, and memory usage. This efficiency is particularly noteworthy given the common assumption that more complex models like GANs would incur higher computational costs. The reduced resource requirements of the GRU–GAN model have significant implications for real-world applications, enabling more frequent model updates, faster threat detection, and deployment in resource-constrained environments.

These findings underscore the potential of our proposed model as a reliable tool for malware detection, especially in dynamic and evolving threat landscapes where new, previously unseen patterns may emerge. The confusion matrix results and prediction performance visualizations further emphasize the effectiveness of our model. While the BiLSTM and BiGRU models showed increasing overlap between malicious and benign classifications across the datasets, the GRU–GAN maintained a clear separation, minimizing both false positives and false negatives. This ability to distinguish between malicious and benign samples with high accuracy is crucial in real-world malware detection systems, where misclassifications can lead to security vulnerabilities or unnecessary alerts. The resilience of the GRU–GAN model to dataset variations suggests its potential effectiveness in detecting novel or evolving malware threats. As malware authors continually develop new techniques to evade detection, a model that can maintain high accuracy across diverse datasets is particularly valuable.

The application of GRU-GAN models in malware detection has shown significant promise, offering advanced capabilities for analyzing API call sequences and detecting malicious patterns. However, despite these advances, several challenges remain unresolved, particularly in the areas of generalization, adversarial robustness, and dynamic adaptation. As malware continues to evolve rapidly and attackers develop more sophisticated techniques, improving the performance and reliability of GRU-GAN models becomes increasingly critical.

One of the key open issues is the challenge of improving generalization in GRU-GAN models for malware detection. The constant evolution of malware families makes it difficult for these models to maintain high accuracy when exposed to novel malware samples not seen during training. GRU-GAN models, while effective with known data, may struggle with new malware behaviors that differ significantly from their training sets. Addressing this issue requires exploring several research directions. One possible approach is to develop advanced data augmentation techniques using GANs to generate diverse and realistic synthetic malware samples. This would help expose models to a wider variety of potential malware behaviors during training.

Additionally, integrating transfer learning techniques could enable the models to adapt more rapidly to new malware families. Another promising direction is the design of meta-learning algorithms, which could improve the model's ability to generalize from limited examples of new malware types. Furthermore, ensemble methods, where multiple GRU-GAN models are trained on different subsets of malware families, could enhance generalization and robustness across a broader range of threats.

Another pressing issue is the adversarial robustness of GRU-GAN models in malware detection. As the use of these models becomes more widespread, they are likely to face intentional attempts to evade detection, with malware authors developing more sophisticated evasion techniques. Ensuring robustness against adversarial attacks is crucial to the long-term effectiveness of GRU-GAN-based detection systems. To improve resilience, researchers could explore adversarial training techniques tailored to the unique characteristics of API call sequences and the GRU-GAN architecture. This would involve generating adversarial examples that simulate potential evasion strategies and incorporating them into the training process. Moreover, developing detection mechanisms for identifying adversarial inputs, perhaps through latent space analysis in the GAN or by examining the confidence levels of the GRU-based discriminator, could further enhance robustness.

A third major challenge involves dynamic adaptation in GRU-GAN models. Given the rapidly evolving nature of cyber threats, it is essential to develop models that can adapt to new malware variants and changing API call patterns without requiring frequent retraining. Retraining models are often computationally expensive and time-consuming, making it impractical for real-time security applications. One promising direction for addressing this challenge is to investigate online learning techniques that allow the model to update its parameters incrementally as new data becomes available. This approach would involve designing specialized loss functions and optimization algorithms that balance the retention of previously learned knowledge with the incorporation of new patterns. Finally, developing efficient techniques for identifying when adaptation is necessary, such as drift detection algorithms or monitoring the model's real-time performance metrics, could help in creating more responsive and dynamic malware detection systems.

5. Conclusions

This paper introduced a novel hybrid deep learning model that combines Gated Recurrent Units (GRUs) and Generative Adversarial Networks (GANs) to enhance the detection of malware by analyzing API call sequences. The GRU-GAN model demonstrated exceptional performance across multiple datasets, achieving a remarkable accuracy of 99.99%, significantly surpassing other models like BiLSTM and BiGRU. In addition to its high accuracy, the GRU-GAN model exhibited robust generalization capabilities, with a precision of 99.7%, a recall of 99.8%, and an AUC of 99% on challenging test datasets.

Furthermore, it maintained low false-positive and false-negative rates, which are critical for minimizing misclassification in real-world malware detection scenarios. This study made some key contributions to the field of malware detection. Firstly, we propose a novel hybrid model that leverages the sequential processing capabilities of GRUs with the generative potential of GANs. This combination allows for more effective detection of sophisticated and evasive malware patterns, improving detection accuracy while maintaining computational efficiency. Secondly, the GRU–GAN model was evaluated against the BiLSTM and BiGRU models using three distinct datasets, demonstrating its superior performance in terms of both accuracy and computational efficiency. It consistently outperformed the other models, showing excellent results in terms of precision, recall, and F1 score, proving its robustness in identifying previously unseen malware samples. Lastly, the GRU–GAN model not only achieved high accuracy but also proved to be computationally efficient. It required significantly less training time (57 s) and memory usage (75 MB) compared to other models, making it highly suitable for real-world applications that demand low resource consumption and quick detection times.

While the current research demonstrated the effectiveness of the GRU–GAN model across three datasets, future work could greatly benefit from incorporating more diverse and comprehensive data sources. One key direction would be to scale the datasets to include a broader spectrum of malware families, including rare or emerging types. By doing so, the model would encounter a wider variety of behaviors during training, thus improving its ability to generalize and detect previously unseen malware. Additionally, collecting datasets from different sectors, such as health care or finance, could offer valuable insights into how malware targeting specific industries operates. This would allow for the development of more specialized detection models. Integrating real-time data, where malware samples are continually fed into the system as they are discovered, would also enable the model to stay current and better adapt to the fast-evolving nature of cyber threats.

Though API call sequences provide critical insights into malware behavior, relying solely on these data could limit the detection capabilities of the model. Future research could explore the integration of additional features, such as file signatures and network traffic data. For example, malware often manipulates file structures or system files, and incorporating file signatures could allow for the detection of these modifications. Network traffic data are also a valuable source of information, as many types of malware establish communication with command-and-control (C2) servers or exfiltrate sensitive data. Monitoring network traffic, alongside API calls, could give a more comprehensive picture of malware activities. System-level metadata, such as file entropy or memory usage patterns, could also be integrated to uncover subtle signs of malicious behavior that API calls alone might not reveal. By enriching the feature set, future models could achieve even higher detection accuracy and resilience against sophisticated threats.

As malware developers continue to innovate with sophisticated evasion techniques, enhancing the robustness of the GRU–GAN model against these strategies becomes an important area for future research. Adversarial training, where adversarial examples are generated to simulate potential evasion strategies, could be incorporated into the model's training process. This would enable the model to become more resilient against attempts to bypass detection. Additionally, the model could be refined to recognize obfuscated API calls or disguised behaviors, which are often employed by malware to evade security systems. Another promising direction would be to focus on defending against polymorphic and metamorphic malware, which alter their code structure frequently to avoid detection. By equipping the GRU–GAN model with mechanisms to identify such variations, future systems could remain effective against an even wider range of evolving malware threats.

The rapid evolution of cyber threats makes it impractical to rely on frequent and computationally expensive retraining of detection models. Future research could focus on developing mechanisms for real-time adaptation, enabling models like GRU–GAN to update incrementally as new data become available without the need for complete retraining. This could be achieved through online learning techniques, where models continually

update their parameters as they encounter new malware patterns. Specialized loss functions and optimization algorithms could be designed to balance the retention of previously learned knowledge with the incorporation of new information. Additionally, research could explore the use of drift detection algorithms or real-time performance monitoring to identify when adaptation is necessary. By developing these adaptive capabilities, future GRU–GAN models could remain effective and responsive in dynamic environments, maintaining their ability to detect emerging threats without sacrificing performance or efficiency.

Author Contributions: N.O. conceived and designed the experiments and wrote the paper. J.A. performed the experiments. S.H. and M.A. reviewed the drafts of the paper. J.O. and A.Q. reviewed and worked on the tables and figures. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets used are publicly available. The API-Call-Sequences dataset [37], the MalbehavD-V1 dataset [38], and the APIMDS dataset [31].

Conflicts of Interest: The authors state that they do not have any competing financial interests or personal relationships that could potentially create biases or otherwise influence the research presented in this paper. The authors affirm that the work reported is free of any competing interests that could undermine the objectivity, integrity, or perceived validity of the paper.

References

1. Han, R.; Kim, K.; Choi, B.; Jeong, Y. A study on detection of malicious behavior based on host process data using machine learning. *Appl. Sci.* **2023**, *13*, 4097. [CrossRef]
2. Almaleh, A.; Almushabb, R.; Ogran, R. Malware API Calls Detection Using Hybrid Logistic Regression and RNN Model. *Appl. Sci.* **2023**, *13*, 5439. [CrossRef]
3. AV-TEST. Malware Statistics & Trends Report. Available online: <https://www.av-test.org/en/statistics/malware/> (accessed on 7 June 2024).
4. Cannarile, A.; Carrera, F.; Galantucci, S.; Iannaccone, A.; Pirlo, G. A Study on Malware Detection and Classification Using the Analysis of API Calls Sequences Through Shallow Learning and Recurrent Neural Networks. In Proceedings of the Italian Conference on Cybersecurity ITASEC22, Rome, Italy, 20–23 June 2022; pp. 124–134.
5. Gibert, D.; Mateu, C.; Planes, J. The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *J. Netw. Comput. Appl.* **2020**, *153*, 102526. [CrossRef]
6. Gençaydin, B.; Kahya, C.N.; Demirkiran, F.; Düzgün, B.; Çayır, A.; Dağ, H. Benchmark Static API Call Datasets for Malware Family Classification. In Proceedings of the 2022 7th International Conference on Computer Science and Engineering (UBMK), Diyarbakir, Turkey, 14–16 September 2022; pp. 1–5.
7. Aslan, Ö.A.; Samet, R. A comprehensive review on malware detection approaches. *IEEE Access* **2020**, *8*, 6249–6271. [CrossRef]
8. Sikorski, M.; Honig, A. *Practical Malware Analysis: The Hands-on Guide to Dissecting Malicious Software*; No Starch Press: San Francisco, CA, USA, 2012.
9. Sihwail, R.; Omar, K.; Ariffin, K.Z. A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis. *Int. J. Adv. Sci. Eng. Inf. Technol.* **2018**, *8*, 1662–1671. [CrossRef]
10. Yousuf, M.I.; Anwer, I.; Riasat, A.; Zia, K.T.; Kim, S. Windows malware detection based on static analysis with multiple features. *PeerJ Comput. Sci.* **2023**, *9*, e1319. [CrossRef] [PubMed]
11. de Oliveira, A.S.; Sassi, R.J. Behavioral malware detection using deep graph convolutional neural networks. *Authorea Prepr.* **2023**, 1–17. [CrossRef]
12. Network, M.D. DeleteFileA Function (fileapi.h). Available online: <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-deletefilea> (accessed on 17 June 2024).
13. Doe, J.; Smith, J. Analyzing API Calls for Legitimacy in Malware Detection. *J. Cyber Secur.* **2022**, *15*, 305–320.
14. Catak, F.O.; Yazı, A.F.; Elezaj, O.; Ahmed, J. Deep learning based Sequential model for malware analysis using Windows exe API Calls. *PeerJ Comput. Sci.* **2020**, *6*, e285. [CrossRef]
15. Sharma, N.; Sharma, R.; Jindal, N. Machine learning and deep learning applications-a vision. *Glob. Transit. Proc.* **2021**, *2*, 24–28. [CrossRef]
16. Yuan, S.; Wu, X. Deep learning for insider threat detection: Review, challenges and opportunities. *Comput. Secur.* **2021**, *104*, 102221. [CrossRef]
17. Pinhero, A.; Anupama, M.; Vinod, P.; Visaggio, C.A.; Aneesh, N.; Abhijith, S.; AnanthaKrishnan, S. Malware detection employed by visualization and deep neural network. *Comput. Secur.* **2021**, *105*, 102247. [CrossRef]
18. Gupta, I.; Kumari, S.; Jha, P.; Ghosh, M. Leveraging LSTM and GAN for Modern Malware Detection. *arXiv* **2024**, arXiv:2405.04373.

19. McLaren, R.A.; Babaagba, K.O.; Tan, Z. A Generative Adversarial Network Based Approach to Malware Generation Based on Behavioural Graphs. In Proceedings of the International Conference on Machine Learning, Optimization, and Data Science, Certosa di Pontignano, Italy, 18–22 September 2022; pp. 32–46.
20. Khan, F.B.; Durad, M.H.; Khan, A.; Khan, F.A.; Rizwan, M.; Ali, A. Design and Performance Analysis of an Anti-Malware System based on Generative Adversarial Network Framework. *IEEE Access* **2024**, *12*, 27683–27708. [\[CrossRef\]](#)
21. Brophy, E.; Wang, Z.; She, Q.; Ward, T. Generative adversarial networks in time series: A systematic literature review. *ACM Comput. Surv.* **2023**, *55*, 199. [\[CrossRef\]](#)
22. Asemi, H. A Study On API Security Pentesting. Master's Thesis, Cal Poly, San Luis Obispo, CA, USA, 2023.
23. Ye, Y.; Li, T.; Adjeroh, D.; Iyengar, S.S. A survey on malware detection using data mining techniques. *ACM Comput. Surv. (CSUR)* **2017**, *50*, 1–40. [\[CrossRef\]](#)
24. Geewax, J.J. *API Design Patterns*; Simon and Schuster: New York, NY, USA, 2021.
25. Vardi, G. On the implicit bias in deep-learning algorithms. *Commun. ACM* **2023**, *66*, 86–93. [\[CrossRef\]](#)
26. Gopinath, M.; Sethuraman, S.C. A comprehensive survey on deep learning based malware detection techniques. *Comput. Sci. Rev.* **2023**, *47*, 100529.
27. Rafique, M.F.; Ali, M.; Qureshi, A.S.; Khan, A.; Mirza, A.M. Malware classification using deep learning based feature extraction and wrapper based feature selection technique. *arXiv* **2019**, arXiv:1910.10958.
28. Brown, A.; Gupta, M.; Abdelsalam, M. Automated machine learning for deep learning based malware detection. *Comput. Secur.* **2024**, *137*, 103582. [\[CrossRef\]](#)
29. Maniriho, P.; Mahmood, A.N.; Chowdhury, M.J.M. API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques. *J. Netw. Comput. Appl.* **2023**, *218*, 103704. [\[CrossRef\]](#)
30. Liu, Y.; Wang, Y. A robust malware detection system using deep learning on API calls. In Proceedings of the 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Chengdu, China, 15–17 March 2019; pp. 1456–1460.
31. Ki, Y.; Kim, E.; Kim, H.K. A novel approach to detect malware based on API call sequence analysis. *Int. J. Distrib. Sens. Netw.* **2015**, *11*, 659101. [\[CrossRef\]](#)
32. Morato, D.; Berrueta, E.; Magaña, E.; Izal, M. Ransomware early detection by the analysis of file sharing traffic. *J. Netw. Comput. Appl.* **2018**, *124*, 14–32. [\[CrossRef\]](#)
33. Hwang, J.; Kim, J.; Lee, S.; Kim, K. Two-stage ransomware detection using dynamic analysis and machine learning techniques. *Wirel. Pers. Commun.* **2020**, *112*, 2597–2609. [\[CrossRef\]](#)
34. Lajevardi, A.M.; Parsa, S.; Amiri, M.J. Markhor: Malware detection using fuzzy similarity of system call dependency sequences. *J. Comput. Virol. Hacking Tech.* **2022**, *18*, 81–90. [\[CrossRef\]](#)
35. Mazaed Alotaibi, F.; Fawad. A Multifaceted Deep Generative Adversarial Networks Model for Mobile Malware Detection. *Appl. Sci.* **2022**, *12*, 9403. [\[CrossRef\]](#)
36. Suaboot, J.; Tari, Z.; Mahmood, A.; Zomaya, A.Y.; Li, W. Sub-curve HMM: A malware detection approach based on partial analysis of API call sequences. *Comput. Secur.* **2020**, *92*, 101773. [\[CrossRef\]](#)
37. Angelo, O. *Malware Analysis Datasets: API Call Sequences*. IEEE Dataport; IEEE: Piscataway, NJ, USA, 2019.
38. Maniriho, P. MalbehavD-V1: A new Dataset of API calls extracted from Windows PE files of benign and malware. Available online: <https://github.com/mpasco/MalbehavD-V1> (accessed on 1 June 2022).
39. Cheng, S. *Recurrent Neural Networks*; Deep Learning Lecture 6; University of Oklahoma: Norman, OK, USA, 2023.
40. Cahuantzi, R.; Chen, X.; Güttel, S. A comparison of LSTM and GRU networks for learning symbolic sequences. In Proceedings of the Science and Information Conference, London, UK, 13–14 July 2023; pp. 771–785.
41. Putpuek, N.; Putpuek, A.; Sungthong, A. A Comparative Study of LSTM, GRU, BiLSTM and BiGRU to Predict Dissolved Oxygen. In Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE), Phitsanulok, Thailand, 28 June–1 July 2023; pp. 67–72.
42. Hu, Z.; Liu, G.; Xiang, X.; Li, Y.; Zhuang, S. GSB: GNGS and SAG-BiGRU network for malware dynamic detection. *PLoS ONE* **2024**, *19*, e0298809. [\[CrossRef\]](#)
43. Goodfellow, I.; Pouget-Abadie, J.; Mirza, M.; Xu, B.; Warde-Farley, D.; Ozair, S.; Courville, A.; Bengio, Y. Generative adversarial nets. *Adv. Neural Inf. Process. Syst.* **2014**, *27*, 1–15.
44. Dunmore, A.; Jang-Jaccard, J.; Sabrina, F.; Kwak, J. Generative adversarial networks for malware detection: A survey. *arXiv* **2023**, arXiv:2302.08558.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.